BiFusion-YOLO: A Lightweight P2-Enhanced YOLO26 with Bidirectional Multi-Scale Fusion for Ultra-Small UAV Detection

Mohamed Ali
Department of Computer Science
Mansoura University
Mansoura, Egypt
mohamedalieladl@gmail.com

Mahmoud M. Safan
Department of Computer Science
Mansoura University
Mansoura, Egypt
saafan2007@mans.edu.eg

Sarah M. Ayyad
Department of Computer Science
Mansoura University
Mansoura, Egypt
sarah_ayyad@mans.edu.eg

Abstract

Long-range drone detection is, at bottom, a resolution problem. A quadcopter seen from a few hundred metres covers fewer than 20×20 pixels, and a detector whose finest prediction map sits at stride 8 has already thrown away most of the evidence it needs. BiFusion-YOLO is a lightweight detector built around that observation. Its backbone pairs Ghost convolution and RepVGG blocks with Coordinate Attention and Haar wavelet downsampling, so that high-frequency detail survives each reduction in resolution; its neck couples a weighted BiFPN with cross-scale feature-fusion blocks; and a dedicated stride-4 head, preceded by a small spatial-attention module, takes over the sub-20-pixel regime while the stride-32 head is dropped. On the DUT Anti-UAV test split the model reaches 97.2% precision, 90.3% recall, 95.6% mAP@0.5 and 66.4% mAP@0.5:0.95 with 1.78M parameters, which is 4.3, 5.0, 4.5 and 5.6 points above YOLO26n. It does so with 25.7% fewer parameters than the 2.40M baseline, at the cost of 34.5% more FLOPs (7.4 vs 5.5 GFLOPs): resolution is bought where the targets are and paid for at stride 32. For targets narrower than 10 pixels, mAP@0.5:0.95 rises from 38.5% to 54.1%, whereas objects wider than 32 pixels lose between 6.5 and 13.0 points; the aggregate still improves because 74.1% of the annotated boxes are narrower than 32 pixels. Taken together, the cumulative ablation, transfer, quantization and error analysis support the combined design; they do not isolate resolution allocation as the sole cause of the gain. The detector runs at 89 FPS in TensorRT FP32 on an RTX 3090 and at 34 FPS in FP16 on a Jetson Orin.

Keywords: Anti-UAV · Ultra-small object detection · YOLO26 · Bidirectional feature pyramid · Haar wavelet downsampling · Coordinate attention · Edge deployment

1 Introduction

Drones have quietly become infrastructure. They spray crops, deliver medical samples, inspect power lines and, increasingly, carry cameras into airspace that nobody agreed to share [1]. The same properties that make them convenient, namely small size, low cost and the ability to launch from almost anywhere, are exactly what makes them awkward for the authorities responsible for airports, prisons and military sites [2, 3, 4]. The Gatwick incident of 2018 is the usual example, but there have been many smaller ones: a drone over a stadium, a bag of contraband over a prison wall, a quadcopter hovering above a chemical plant. Reliable detection at range is the first requirement for any counter-UAV (C-UAV) system, and in most deployments it has to run on a camera at the edge, close to where the video is captured, because shipping many streams to a central server is expensive and often impossible.

Radar was the obvious answer until people tried it. Small composite airframes have a tiny radar cross-section, hover with almost no radial velocity and disappear into ground clutter [5, 6]. Radio-frequency sensing works when the pilot is transmitting, but a drone flying a pre-programmed route emits nothing [7], and acoustic arrays are cheap but deaf beyond a short range in any wind [8]. Cameras avoid all three problems. They are passive, they resolve appearance rather than motion, and they are already installed on the infrastructure that needs protecting [9, 10]. Benchmarks built for exactly this setting now exist [11, 12, 13], which is one reason the visible-spectrum detector has become the workhorse of camera-based C-UAV pipelines.

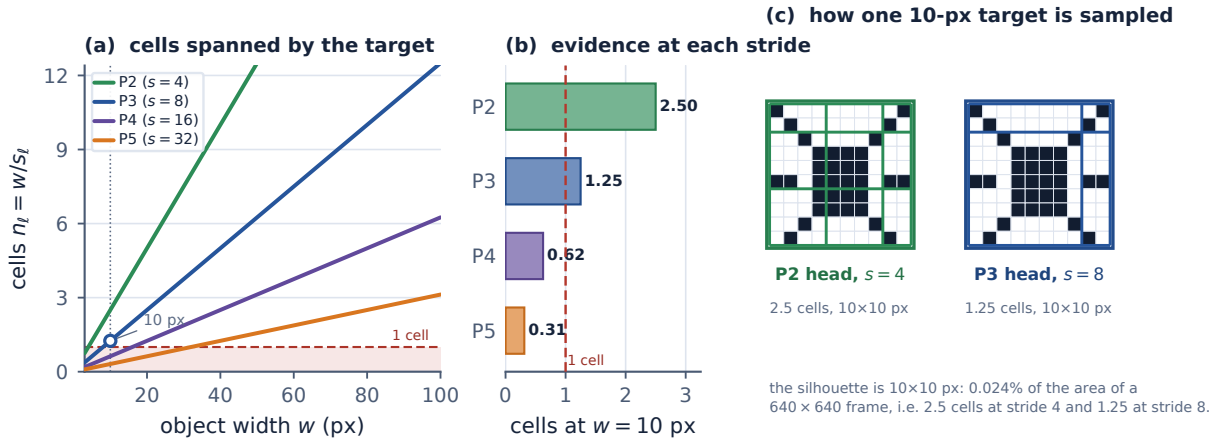


Figure 1. Motivation for resolution-aware detection. (a) Number of feature cells spanned by an object of width w at each detection stride; the shaded band marks the sub-cell regime, in which a target is represented by less than one cell and the classification branch has almost no spatial support. (b) Cells covered by a 10-pixel UAV at each stride. (c) The same 10×10-pixel silhouette sampled on the P2 (stride 4) and P3 (stride 8) grids: the proposed P2 head gives 2.5 cells, whereas the finest baseline head gives 1.25.

The difficulty is that a drone at distance is almost not an object. In the DUT Anti-UAV detection set more than half of the annotated boxes are narrower than 20 pixels, the average object covers about 1.3% of the frame, and the smallest target occupies 1.9×10^{-6} of the image [12]: a handful of pixels, possibly darker or lighter than the sky behind them, and nothing more. Scale changes by more than an order of magnitude as the drone approaches. Birds look similar at the same distance, and so do window frames, roof edges and insulators. Meanwhile a C-UAV station may have to watch four or eight streams at once, which rules out anything that needs a data-centre GPU.

Single-stage detectors are the natural fit for that budget, and the YOLO family has been the default choice for a decade [14, 15, 16, 17, 18, 19]. The trouble is that these architectures were tuned for objects that fill a reasonable share of the input. In YOLO26 the finest head reads from the P3 map at stride 8, so a 10-pixel UAV is worth 1.25 feature cells; at P4 it is 0.63 and at P5 it is 0.31. By the time the feature reaches the head, a strided convolution has already acted as a low-pass filter and the evidence that the object existed is mostly gone. Feeding a larger image helps a little and costs a lot, since the compute grows with the square of the resolution, and it does nothing about the aliasing introduced by the downsampling itself. Two further habits of generic detectors make things worse: fusion across scales is usually an unweighted sum, and channel attention is usually global, which averages away the very position information that separates a distant drone from a distant chimney.

Our position is that this regime needs the backbone, the neck and the heads to be redesigned together rather than patched with a single module. BiFusion-YOLO is the result. A P2 head at stride 4 opens a high-resolution path to the predictor. Haar wavelet downsampling replaces the strided convolutions on that path, so the sub-bands that carry edges and corners are kept instead of averaged out. Coordinate Attention, which factorizes pooling into a row and a column descriptor, is inserted after each backbone stage to keep directional information alive. The neck combines a weighted BiFPN with cross-scale fusion blocks that align channels before mixing them. To pay for the extra resolution, Ghost convolution and re-parameterized RepVGG blocks keep the backbone thin, and the stride-32 head is removed altogether.

Figure 1 makes the arithmetic of this choice explicit. At the stride used by the finest baseline head a 10-pixel target is barely one cell wide; at stride 4 the same target covers two and a half cells, and the regression branch finally has something to interpolate over.

The contributions of the paper can be summarized as follows.

- **A resolution-aware design, not a module swap.** We turn the pixel-size/stride relationship into a concrete design rule and follow it through: a stride-4 head, a detail-preserving downsampling path feeding it, and a thinner network elsewhere to pay for it. The resulting detector is lighter than the YOLO26n baseline while being markedly better on the targets that matter.
- **Weighted bidirectional fusion with channel alignment.** A weighted BiFPN is combined with cross-scale fusion blocks that project every level to a common width before mixing. The pair adds 0.08M parameters to the PAN-FPN it replaces, which is more than repaid by removing the stride-32 head. The cumulative ablation evaluates the incremental effect of progressively adding the proposed components in the selected architectural order, while acknowledging that component interactions are not fully isolated.

- **A backbone built for small, low-contrast evidence.** Ghost convolution, RepVGG re-parameterization and Coordinate Attention are arranged in stages, and controlled comparisons against SE, CBAM and ECA show that Coordinate Attention is the variant that pays off for tiny targets.
- **An evaluation that looks past the aggregate score.** Besides the usual comparison table we report size-stratified AP, parameter/FLOP/latency trade-offs against three quantization levels on two devices, direct and few-shot transfer to three other datasets, ten-seed significance tests, and a breakdown of every remaining error. The headline gains are +4.5 mAP@0.5 and +5.6 mAP@0.5:0.95, with 25.7% fewer parameters and 34.5% more FLOPs than the baseline; we report the FLOP increase rather than hiding it.

The rest of the paper follows the usual order: related work (Section 2), a formal statement of the problem and an analysis of the baseline (Section 3), the architecture and training objective (Section 4), the experimental protocol (Section 5), results (Section 6), and a discussion of what the numbers do and do not support (Section 7).

2 Related Work

2.1 Non-Visual and Multi-Modal UAV Detection

Counter-UAV sensing began with radar, and radar still does the long-range job when the target is cooperative. Small airframes are not. Their radar cross-section is weak, their radial velocity is close to zero in a hover, and micro-Doppler signatures are easily buried in clutter [5, 6]. Passive bistatic receivers and RF fingerprinting can sometimes catch the control or video link, but they are silent against a drone flying a stored route [7]. Acoustic arrays are attractive on price and poor on range, particularly in any wind [8]. Fusing several of these modalities buys robustness at the cost of calibration and cost of ownership [3], and the infrared branch is often added to cover the hours when the visible camera is useless; recent work in that direction worries about misalignment between the two streams and routes features through a sparse mixture of experts [20]. Our work sits downstream of all this. We take the visible-spectrum detector that most camera-based systems already run and try to make it see further.

2.2 Generic Object Detectors

Detectors divide conveniently into two-stage, one-stage and transformer families. Faster R-CNN [21] and its cascade variants remain accurate and remain hard to run at 30 FPS on an embedded board. SSD [22], RetinaNet [23], FCOS [24] and the YOLO line [14, 15, 16, 25, 26, 17, 27, 28, 18, 19] cover the real-time end of the spectrum. DETR [29], its deformable successor [30] and RT-DETR [31] remove anchors and NMS, but at this scale they are heavier for the same accuracy. YOLO26, the baseline we build on, is a recent member of that family and one of the first to specialise by model size: the compact variants keep a convolutional backbone and predict from three scales (P3, P4 and P5), with a strengthened high-resolution path and foreground-alignment supervision, while the larger ones move to query-based decoding [19, 32, 33].

None of these architectures is badly designed for what it targets. They are designed for objects that occupy something like 1% of the image or more, and every one of them puts its finest head at stride 8 or coarser. For a 10-pixel drone that is the wrong operating point, and no amount of extra capacity fixes a feature map on which the object never had more than one cell.

2.3 Small-Object Detection

Work on small objects has moved along four fronts. Multi-scale representation is the oldest: FPN [34], PANet [35], NAS-FPN [36] and BiFPN [37] all trade some computation for better cross-scale flow, and high-resolution heads or image pyramids push the same idea further at a cost that is usually quadratic. Attention is the second: SE [38], CBAM [39], ECA [40], SimAM [41] and Coordinate Attention [42] recalibrate responses, with Coordinate Attention unusual in keeping separate row and column descriptors instead of collapsing both dimensions. Lightweight design is the third, and MobileNetV2 [43], ShuffleNetV2 [44], GhostNet [45] and RepVGG [46] are the usual building blocks. The fourth front is data: copy-paste augmentation of small objects [47], multi-scale training and context-aware augmentation [48]. Haar wavelet downsampling [49] belongs to a fifth, smaller line that attacks the downsampling operator itself, replacing a lossy stride with a transform that is at least invertible in principle.

2.4 UAV-Specific Detectors and Benchmarks

The DUT Anti-UAV benchmark [12] supplies 10,000 visible-light images with 10,109 annotated UAVs and has become the default testbed for anti-UAV detection. The Anti-UAV benchmark [11] adds multi-modal tracking sequences, and Drone-vs-Bird [13] is built around the discrimination problem that matters most in the field. Papers built on these datasets tend to fall into three groups. The first reworks the neck and head of a YOLO backbone: Gao *et al.* swap strided downsampling for HWD, rebuild the neck with CCFM and add a P2 head, reaching 95.2% mAP@0.5 with 1.59M

Table 1. Positioning with respect to representative lightweight UAV detectors. A checkmark indicates that the design explicitly includes the corresponding ingredient; a blank cell indicates that it does not. HR = high-resolution head, Bi = bidirectional fusion, Attn = spatial or coordinate attention, Light = an explicit lightweighting strategy. Accuracy values are quoted from the original papers and are *not* directly comparable, because datasets and protocols differ; they are included only to situate the operating points. Where a paper reports only mAP@0.5, the strict metric is omitted.

Method	Year	Params (M)	mAP@0.5 (%)	Dataset	HR	Bi	Attn	Light
YOLO26n [19]	2026	2.40	91.1	DUT			✓	✓
Gao *et al.* [50]	2025	1.59	95.2	DUT	✓			✓
Yolov5-CBi [51]	2026	17.76	95.45	DUT		✓	✓	
LER-YOLO [20]	2026	12.21	89.7	MBU			✓	
SDD-YOLO-n [54]	2026	2.50	86.0	DroneSOD	✓		✓	✓
CSCWD [53]	2026	2.58	50.06	DUT (ZS)	✓		✓	✓
BiFusion-YOLO (ours)	2026	1.78	**95.6**	DUT	✓	✓	✓	✓

DUT = DUT Anti-UAV; ZS = zero-shot transfer from another domain; MBU = multi-band UAV set; DroneSOD = DroneSOD-30K. The method is named Yolov5-CBi in the source paper, whose title is YolovN-CBi. On DUT Anti-UAV, Gao *et al.* additionally report 64.9% mAP@0.5:0.95; the remaining entries report mAP@0.5 only.

parameters [50]. The second leans on attention or multi-modal fusion: Yolov5-CBi combines CBAM with BiFPN [51], LER-YOLO routes RGB and infrared features through a reliability-guided mixture of experts [20], and dual-stream designs target night operation and payload identification [52]. The third optimizes for deployment, either by distilling cross-scale knowledge into a YOLO11n student [53] or by pairing a P2 head with a DFL-free, NMS-free pipeline [54]. Multi-frame and tiled inference [55, 56] and camouflage-aware fusion [57, 58] have also been explored.

2.5 Research Gap

Three gaps are worth naming. First, the papers that add a high-resolution head and the papers that redesign the fusion path tend to be different papers; where both appear, the interaction between them is not isolated, so it is hard to say which one is responsible for the gain. Second, HWD has mostly been used as a drop-in replacement for a strided convolution. Nobody, as far as we can tell, has measured what it does when the sub-bands it preserves are actually consumed by a stride-4 head, which is the configuration in which they should matter most. Third, the UAV literature reports on different datasets and input protocols, so the comparison tables in individual papers say less than they appear to.

Table 1 puts the resulting landscape in one place. Among detectors below 2M parameters, we did not find one in the literature we surveyed that combines a P2-level head, detail-preserving downsampling, weighted bidirectional fusion and positional attention. Ghost convolution, RepVGG, Coordinate Attention, wavelet downsampling, BiFPN and cross-scale fusion are all established modules, and none of them originates here; what the paper contributes is the combination, the adaptation of the fusion block to this neck, and the controlled evaluation of how the parts interact. That combination is the contribution of this paper, and the ablations are designed to show what each ingredient adds, with parameters, FLOPs, latency, quantization and size-stratified AP all reported so that the claim can be checked rather than trusted.

3 Problem Formulation and Baseline Analysis

3.1 Problem Statement

Let $I \in \mathbb{R}^{H\times W\times 3}$ be an RGB frame and let

$$\mathcal{B} = \{(b_i, c_i)\}_{i=1}^{M}, \qquad b_i = (x_i, y_i, w_i, h_i), \tag{1}$$

be the set of M annotated UAV boxes, where (x_i, y_i) is the box centre, (w_i, h_i) its width and height, and $c_i \in \{1\}$ the single UAV class. A detector f_θ returns a set of N candidates

$$\hat{\mathcal{B}} = f_\theta(I) = \{(\hat{b}_j, \hat{s}_j)\}_{j=1}^{N}, \tag{2}$$

with $\hat{s}_j \in [0, 1]$ the confidence that candidate j is a UAV. Training amounts to finding

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{(I,\mathcal{B})\sim\mathcal{D}} \left[\mathcal{L}\left(f_\theta(I), \mathcal{B}\right)\right], \tag{3}$$

where $\mathcal{D}$ is the training distribution and $\mathcal{L}$ gathers the classification, regression and distribution-quality terms of Section 4.5.

Table 2. Number of feature cells spanned by a UAV of width w at each detection stride (Eq. (5)). Values below one cell, shown in italics, indicate that the target is sub-sampled by the feature map: the classifier then sees a single activation and the regressor must infer the box extent from it.

Target width	**P2** (s=4)	**P3** (s=8)	**P4** (s=16)	**P5** (s=32)
5 px	1.25	*0.63*	*0.31*	*0.16*
10 px	2.50	1.25	*0.63*	*0.31*
20 px	5.00	2.50	1.25	*0.63*
40 px	10.00	5.00	2.50	1.25
80 px	20.00	10.00	5.00	2.50

We follow the COCO convention [59] and call an object with area $a_i = w_i h_i < 32^2$ px^2 *small*, one with $32^2 \le a_i < 96^2$ px^2 *medium* and one with $a_i \ge 96^2$ px^2 *large*. Its pixel occupancy is

$$\rho_i = \frac{a_i}{HW}. \tag{4}$$

On the DUT Anti-UAV detection set the mean occupancy is about 1.3% and the smallest object reaches $\rho \approx 1.9 \times 10^{-6}$ [12]. At that size there is very little left to recognize: a few luminance values, no texture, and often no more than one edge.

3.2 Scale Analysis and the Need for a P2 Head

What a head can see is fixed by its stride. An object of width w spans

$$n_\ell(w) = \frac{w}{s_\ell}, \qquad s_\ell \in \{4, 8, 16, 32\} \tag{5}$$

cells at level ℓ, and Table 2 evaluates this for widths that occur in the benchmark. At P3, the finest level YOLO26 uses, a 10-pixel UAV is worth 1.25 cells; at P4 and P5 it is worth less than one. Below one cell the classifier is looking at a single activation and the regressor has to invent an extent from it, which is where the very loose boxes on distant targets come from. Moving the same target to stride 4 gives 2.5 cells, and a 20-pixel target 5 cells, enough for both tasks to become meaningful.

The design rule that follows is blunt: if a meaningful share of the targets is narrower than 20 pixels, the detector should show a stride-4 response to the head. Empirically this is what the P2-head papers in aerial detection find as well [53, 54]. The catch is cost. A P2 map has four times the spatial area of P3, and both the head and the fusion path grow with it, so adding the head without thinning anything else would grow the parameter count and, more sharply, the compute. Every subsequent design decision in Section 4 is, in one way or another, about paying that bill.

3.3 Baseline: YOLO26

YOLO26 is the 2026 member of the Ultralytics family, and it takes a different approach to scaling than its predecessors: rather than stretching a single design across the whole size range, the compact variants keep a streamlined convolutional network while the larger ones switch to query-based decoding. The nano model we use as the baseline belongs to the convolutional branch [19, 32, 33]. Its backbone is a plain CNN stack with a deliberately strengthened high-resolution path, the neck is a PAN-style fusion network, and prediction is multi-scale: three heads read from strides 8, 16 and 32, that is, from P3, P4 and P5. The term *dual-head* that appears in some descriptions of YOLO26 refers to its pair of one-to-many and one-to-one assignment branches, which let the same weights be decoded either with NMS or through an NMS-free path; it does not mean that the model predicts from two scales. Supervision is dense and includes a foreground-alignment term. Throughout the paper, *YOLO26n* refers to the official nano architecture distributed with Ultralytics, with the stock backbone and neck and with COCO pretrained weights; the training recipe is the common protocol of Section 5.5, not YOLO26's native one. We changed nothing in the baseline architecture itself, only in the model we build on top of it. It has 2.40M parameters and 5.5 GFLOPs at 640×640 input.

Three things about this pipeline are wrong for our target regime, and Figure 2 marks them. Predictions never come from a stride-4 representation. The stage transitions lose high-frequency content through strided convolution and pooling, so whatever detail survived the first stages is gone by P3, and the strengthened high-resolution path of YOLO26n does not change that, since its finest prediction still sits at stride 8. And the neck fuses levels with fixed, equal weights and no channel alignment, which lets a high-channel deep feature swamp the one coming from P2. None of this is a defect in YOLO26 for the objects it was designed for; it is simply the wrong set of choices for a target that is smaller than the stride.

3.4 Design Requirements

Those observations translate into five requirements.

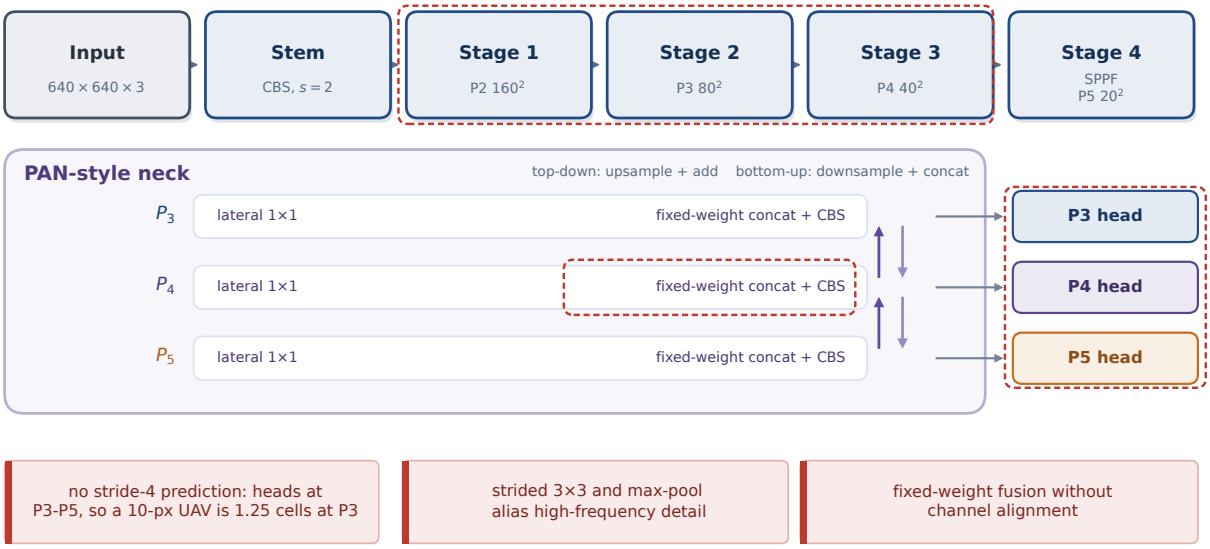


Figure 2. The YOLO26n baseline and the three properties that limit ultra-small UAV detection: ① prediction is multi-scale but never at stride 4; the heads sit at P3, P4 and P5, so the finest head operates at $1/8$ and a 10-pixel UAV spans 1.25 cells; ② the resolution-reducing stages use strided convolution and pooling, which act as low-pass filters and alias high-frequency detail; ③ PAN-style fusion combines levels with fixed weights and without channel alignment. Dashed red boxes mark the criticised elements.

R1: High-resolution evidence.
At least one head must read from a stride-4 map, so that 8-20-pixel targets span more than one cell (Table 2).

R2: Downsampling that keeps detail.
Resolution reduction should carry the high-frequency sub-bands forward instead of aliasing them away, in particular at the P2→P3 and P3→P4 transitions.

R3: Fusion that does not cost the earth.
P2-P5 must exchange semantic and positional information without the quadratic cost of dense convolutions on the high-resolution map.

R4: Attention that knows where it is.
Recalibration should preserve direction, because global pooling destroys the one cue that distinguishes a distant drone from a distant piece of background.

R5: Real-time operation.
The finished model should be smaller than the baseline and hold at least 30 FPS on an embedded GPU, otherwise it cannot serve more than one camera.

4 Proposed Method

4.1 Overall Architecture

Figure 3 gives the full picture. The network keeps the backbone-neck-head arrangement of YOLO26 and changes four architectural things that depend on each other: a thinner backbone that does not throw detail away, a bidirectional neck that refines each scale after fusion, a P2 head, and a small attention module in front of the predictor at that level. The detection head keeps the decoupled, anchor-free form and a 16-bin DFL regression branch; that branch comes from earlier YOLO versions rather than from YOLO26, and Section 4.5 discusses the departure. For an input $I \in \mathbb{R}^{640\times640\times3}$ the backbone produces

$$\{F_2, F_3, F_4, F_5\} = \phi_{\text{backbone}}(I), \tag{6}$$

at 160×160, 80×80, 40×40 and 20×20, that is, at strides 4, 8, 16 and 32. The neck maps them to

$$\{\tilde{F}_2, \tilde{F}_3, \tilde{F}_4, \tilde{F}_5\} = \phi_{\text{neck}}(F_2, F_3, F_4, F_5), \tag{7}$$

and the heads read from $\tilde{F}_2$, $\tilde{F}_3$ and $\tilde{F}_4$. The baseline predicts from three levels, P3, P4 and P5. We keep the two finer levels and add a stride-4 head in place of the stride-32 one, so the network still predicts from three levels, now P2, P3 and P4. Dropping the stride-32 response costs little in this regime, where a UAV at range is never large, and it removes a steady source of large false positives on cloud edges and building silhouettes. It also pays part of the cost of adding P2.

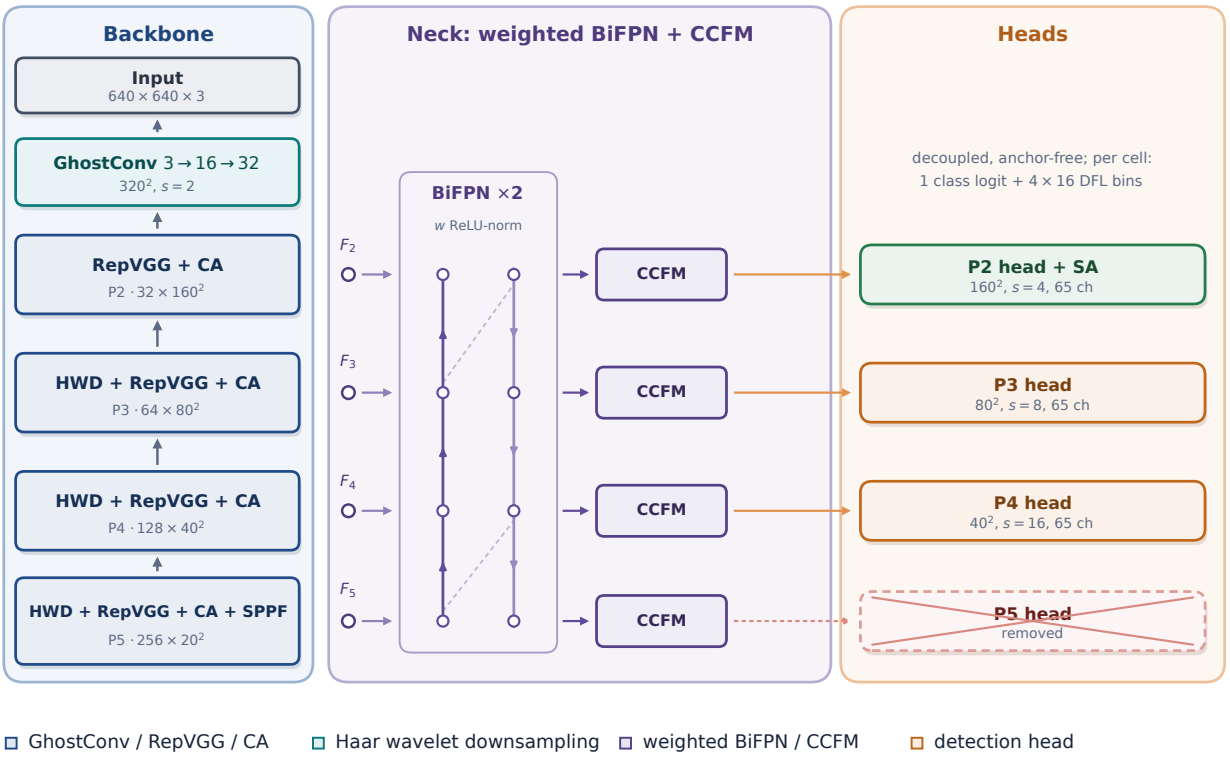


Figure 3. Overall architecture of BiFusion-YOLO. The backbone stack (left) uses Ghost convolution, RepVGG, Coordinate Attention, and Haar wavelet downsampling, and produces the four pyramid levels F_2-F_5 at strides $4/8/16/32$. The neck (middle) couples two weighted BiFPN blocks with per-level CCFM refinement; the fused features feed three decoupled anchor-free heads (right) at P_2, P_3, and P_4. The P_2 branch includes a lightweight spatial-attention module before prediction, and the redundant P_5 head is removed. Each output cell predicts one class logit and four box sides discretised into 16 DFL bins.

4.2 Lightweight Detail-Preserving Backbone

The backbone has to satisfy R1, R2, R4 and R5 at once, which in practice means being generous at high resolution and stingy everywhere else. It starts with a two-step GhostConv stem ($3 \to 16 \to 32$, stride 2 each) and then runs four stages of width $32, 64, 128, 256$, one RepVGG block per stage. Ghost convolution appears only in the stem, where the spatial area is largest and the saving is worth having. Haar wavelet downsampling replaces the strided convolution at all three transitions. Coordinate Attention follows every RepVGG block. The four modules are drawn in Figure 4.

4.2.1 Ghost Convolution

A dense convolution produces many nearly identical feature maps. Ghost convolution [45] computes $m = n/s$ of them with a normal kernel and manufactures the remaining $n - m$ with cheap depth-wise transforms. For input $\mathbf{X} \in \mathbb{R}^{h \times w \times c}$,

$$\mathbf{Y}' = \mathbf{X} * \mathbf{\Phi}, \qquad \mathbf{\Phi} \in \mathbb{R}^{k \times k \times c \times m}, \tag{8}$$

$$\mathbf{Y}''_j = \Psi_j(\mathbf{Y}'), \qquad j = 1, \ldots, s-1, \tag{9}$$

$$\mathbf{Y} = [\mathbf{Y}', \mathbf{Y}''_1, \ldots, \mathbf{Y}''_{s-1}], \tag{10}$$

where Ψ_j is a $d \times d$ depth-wise operation and $[\cdot]$ concatenates along channels. For a layer with c inputs and n outputs the ratio of multiply-accumulate operations to a dense $k \times k$ convolution is

$$r_{\text{Ghost}} = \frac{1}{s} + \frac{s-1}{s} \cdot \frac{d^2}{k^2\, c}. \tag{11}$$

With $s = 2$, $k = 3$, $d = 3$ and $c = 32$ this gives $r_{\text{Ghost}} \approx 0.52$, so roughly half the multiplications disappear at the same output width. The catch is the $1/c$ term: at $c = 256$ the ratio is 0.502 and the saving is negligible. That is why the operation is confined to the stem and the deeper stages keep ordinary RepVGG blocks, where the extra representational strength is worth more than the marginal arithmetic.

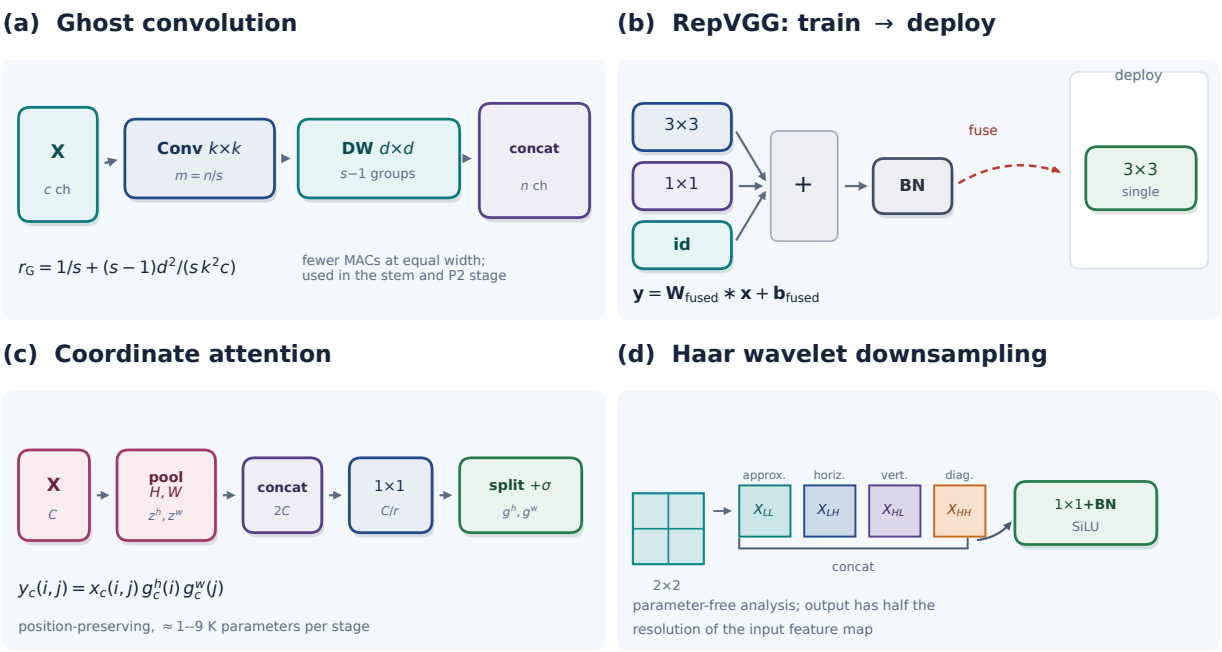


Figure 4. Backbone building blocks. (a) Ghost convolution generates a small set of intrinsic features and augments them with cheap depth-wise operations. (b) RepVGG uses three branches during training and a single 3×3 convolution after re-parameterization. (c) Coordinate Attention encodes height and width position information separately before re-weighting the input. (d) Haar wavelet downsampling decomposes the feature map into four sub-bands and fuses them with a 1×1 convolution, so that high-frequency detail survives the resolution reduction.

4.2.2 RepVGG Blocks

RepVGG [46] gets a three-branch block during training and a single convolution at inference. In training,

$$\mathbf{y} = \mathrm{BN}_{3\times3}(\mathbf{W}_{3\times3} * \mathbf{x}) + \mathrm{BN}_{1\times1}(\mathbf{W}_{1\times1} * \mathbf{x}) + \mathrm{BN}_{\mathrm{id}}(\mathbf{x}), \tag{12}$$

with independent batch-normalization statistics for the three branches. At inference each branch collapses into one 3×3 kernel:

$$\mathbf{y} = \mathbf{W}_{\mathrm{fused}} * \mathbf{x} + \mathbf{b}_{\mathrm{fused}}, \tag{13}$$

where

$$\mathbf{W}_{\mathrm{fused}} = \mathbf{W}_{3\times3} + \mathrm{pad}_{3\times3}(\mathbf{W}_{1\times1}) + \mathrm{pad}_{3\times3}(\mathbf{I}), \tag{14}$$

and the BN scales and biases are absorbed into $\mathbf{b}_{\mathrm{fused}}$. The deployed block therefore costs exactly what a 3×3 convolution costs, which matters on a Jetson, while the training-time branches keep the gradient path short enough for a network of this depth to optimize comfortably.

4.2.3 Coordinate Attention

Global average pooling throws away position, which is precisely the information that separates a ten-pixel drone from a ten-pixel piece of roof. Coordinate Attention (CA) [42] avoids the collapse by pooling along the two axes separately. For channel c,

$$z_h^c(h) = \frac{1}{W} \sum_{0 \le i < W} x_c(h, i), \tag{15}$$

$$z_w^c(w) = \frac{1}{H} \sum_{0 \le j < H} x_c(j, w). \tag{16}$$

The descriptors are concatenated, projected with a shared 1×1 convolution whose bottleneck has $\max(C/32, 8)$ channels, and split again into a height-aware and a width-aware map:

$$[g^h, g^w] = \delta\big(\mathrm{BN}(\mathrm{Conv}_{1\times1}([z^h; z^w]))\big), \tag{17}$$

$$g^h = \sigma(\mathrm{Conv}_{1\times1}(g^h)), \qquad g^w = \sigma(\mathrm{Conv}_{1\times1}(g^w)), \tag{18}$$

with δ a non-linearity and σ the sigmoid. The output is

$$y_c(i, j) = x_c(i, j)\, g_c^h(i)\, g_c^w(j). \tag{19}$$

Because Eq. (19) multiplies by a row factor and a column factor rather than by a single scalar, the module stays spatially selective, and it does so for 1.1-8.7K parameters per stage, under 1% of the model. On tiny targets that selectivity is most of the value: a distant quadcopter often shows up as one short horizontal streak, and a module that can weight rows differently from columns is in a position to keep it.

4.2.4 Haar Wavelet Downsampling

Strided convolution and max pooling are low-pass filters, and a drone at range is high-frequency content. HWD [49] replaces them with a two-dimensional Haar transform. The one-dimensional scaling and wavelet functions are

$$\phi(x) = \frac{1}{\sqrt{2}}\phi_{1,0}(x) + \frac{1}{\sqrt{2}}\phi_{1,1}(x), \tag{20}$$

$$\psi(x) = \frac{1}{\sqrt{2}}\phi_{1,0}(x) - \frac{1}{\sqrt{2}}\phi_{1,1}(x), \tag{21}$$

with

$$\phi_{j,k}(x) = \sqrt{2^j}\,\phi(2^j x - k), \qquad k = 0, 1, \ldots, 2^j - 1. \tag{22}$$

One level of decomposition of a map x yields four sub-bands:

$$X_{LL}(i,j) = \tfrac{1}{2}\big[x(2i,2j) + x(2i,2j+1) + x(2i+1,2j) + x(2i+1,2j+1)\big], \tag{23}$$

$$X_{LH}(i,j) = \tfrac{1}{2}\big[x(2i,2j) + x(2i,2j+1) - x(2i+1,2j) - x(2i+1,2j+1)\big], \tag{24}$$

$$X_{HL}(i,j) = \tfrac{1}{2}\big[x(2i,2j) - x(2i,2j+1) + x(2i+1,2j) - x(2i+1,2j+1)\big], \tag{25}$$

$$X_{HH}(i,j) = \tfrac{1}{2}\big[x(2i,2j) - x(2i,2j+1) - x(2i+1,2j) + x(2i+1,2j+1)\big], \tag{26}$$

where LL is the low-frequency approximation and LH, HL and HH carry horizontal, vertical and diagonal detail. The four are concatenated and fused:

$$\mathbf{Y} = \mathrm{SiLU}(\mathrm{BN}(\mathrm{Conv}_{1\times1}([X_{LL}; X_{LH}; X_{HL}; X_{HH}]))), \tag{27}$$

which halves the spatial resolution and sets the channel count. In our backbone HWD sits at the P2→P3, P3→P4 and P4→P5 transitions. The bookkeeping is favourable, counting one multiply-accumulate per weight-input pair: the 1×1 fusion in Eq. (27) costs about $4C_{\mathrm{in}}C_{\mathrm{out}}$ rather than the $9C_{\mathrm{in}}C_{\mathrm{out}}$ of a 3×3 stride-2 convolution, roughly 56% fewer multiplications, and the wavelet analysis itself has no parameters. The point, though, is not the saving. It is that the LH, HL and HH maps reach P3 and P4 as separate channels, so edge and corner evidence that a strided convolution would have averaged into oblivion is still available when the head needs it. The transform itself only exposes those sub-bands; whether the learned 1×1 fusion makes use of them is an empirical question, and the ablation in Section 6.3 is what answers it.

4.3 Neck: BiFPN with Cross-Scale Feature Fusion

The neck has to move semantic information down from P5 and positional detail up from P2 without the high-resolution branch being drowned by the deep one. We split the job: a weighted BiFPN handles propagation, and a CCFM pass refines each level afterwards. Both are drawn in Figure 5.

4.3.1 Weighted Bidirectional Feature Pyramid

BiFPN [37] removes nodes that have only one input and puts a learnable weight on every fusion. Writing P_i^{in} for the backbone feature at level i and P_i^{td}, P_i^{out} for the intermediate and output features, the two directions are

$$P_i^{td} = \mathrm{Conv}\left(\frac{w_{i,1}P_i^{in} + w_{i,2}\,\mathrm{Resize}(P_{i+1}^{td})}{w_{i,1} + w_{i,2} + \epsilon}\right), \tag{28}$$

$$P_i^{out} = \mathrm{Conv}\left(\frac{w'_{i,1}P_i^{in} + w'_{i,2}P_i^{td} + w'_{i,3}\,\mathrm{Resize}(P_{i-1}^{out})}{w'_{i,1} + w'_{i,2} + w'_{i,3} + \epsilon}\right), \tag{29}$$

where, in Eqs. (28) and (29), the $w \geq 0$ are learnable weights, passed through ReLU to keep them nonnegative and then divided by their sum plus $\epsilon = 10^{-4}$, which is the fast normalized fusion of EfficientDet [37]. The denominator is what makes the weights comparable across levels: it turns each $w_{i,j}$ into a share of the fusion for that node, so the network can say how much of the output should come from the backbone feature and how much from the propagated one, and it can say something different for every node and every frame. Two blocks are stacked, the first doing the propagation and the second cleaning up the result.

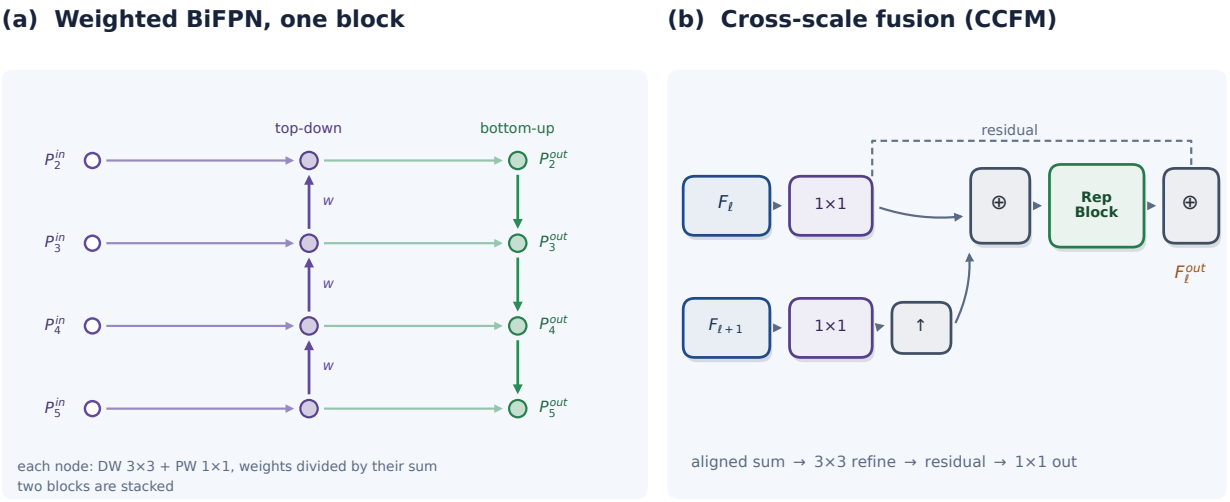


Figure 5. Neck modules. (a) Weighted BiFPN: the top-down path (up-sampling and weighted addition) and the bottom-up path (down-sampling and weighted concatenation) operate on all four levels, with learnable fusion weights w normalized by their sum plus ϵ. (b) CCFM: adjacent scales are channel-aligned by 1×1 convolutions, summed, refined by a RepBlock, and added to the aligned feature before the per-level output projection.

4.3.2 Cross-Scale Feature Fusion

Resizing aligns scales geometrically but not statistically: the channel statistics at P2 and P5 have very little in common, and concatenating them lets the deeper map dominate. CCFM, borrowed in spirit from the RT-DETR hybrid encoder [31], projects both operands to a common width first and refines the sum locally. For adjacent features F_l and F_{l+1},

$$\tilde{F}_l = \mathrm{Conv}_{1\times1}(F_l), \qquad \tilde{F}_{l+1} = \mathrm{Resize}(\mathrm{Conv}_{1\times1}(F_{l+1}))\,, \tag{30}$$

$$F_l^{out} = \mathrm{Conv}_{1\times1}^{(l)}\Big(\mathrm{RepBlock}(\tilde{F}_l + \tilde{F}_{l+1}) + \tilde{F}_l\Big)\,, \tag{31}$$

with the residual term holding on to the original scale and the RepBlock supplying a 3×3 receptive field for local context. Both operands are projected to a common fusion width $C_\mathrm{f} = 64$ before the sum, which is cheap and stops a 256-channel feature from swamping a 32-channel one. The output projection $\mathrm{Conv}_{1\times1}^{(l)}$ then returns level l to the width its consumer expects: 32 channels at P2, 64 at P3 and 128 at P4, while the P5 branch ends inside the neck because it feeds no detection head. We run a single CCFM pass after the two BiFPN blocks, at P2 through P5. The division of labour is clean enough to state in one line: BiFPN decides where information goes, CCFM decides how the channels are mixed once it gets there.

4.4 Detection Heads

4.4.1 P2-Level Head with Spatial Attention

The P2 branch leaves the neck as $\tilde{F}_2 \in \mathbb{R}^{160\times160\times32}$, the fusion width reduced to 32 by the CCFM output projection, and it carries most of the tiny-target evidence. Most of what it carries is background, so a light spatial-attention module suppresses the background before prediction:

$$M_s = \sigma\Big(\mathrm{Conv}_{7\times7}\left([\mathrm{AvgPool}(\tilde{F}_2); \mathrm{MaxPool}(\tilde{F}_2)]\right)\Big)\,, \tag{32}$$

$$P_2^{final} = \tilde{F}_2 \odot M_s, \tag{33}$$

where $[\cdot]$ is again channel concatenation and $\odot$ element-wise multiplication. Average and maximum pooling together give the module both the local mean level and the local peak, and the 7×7 kernel gives it enough context to tell a small bright target from a small bright window frame. The whole module costs 99 parameters. The output goes through a 1×1 projection to 64 channels and then into the decoupled head; P3 and P4 use the same head without the attention stage. Figure 6 shows the branch.

4.4.2 Anchor-Free Decoupled Prediction

Each head emits a classification logit and a box distribution per location. The regression branch predicts the four distances (l, t, r, b) to the box sides, each discretized into $n_{bins} = 16$ bins as in distribution focal loss [60]. Being

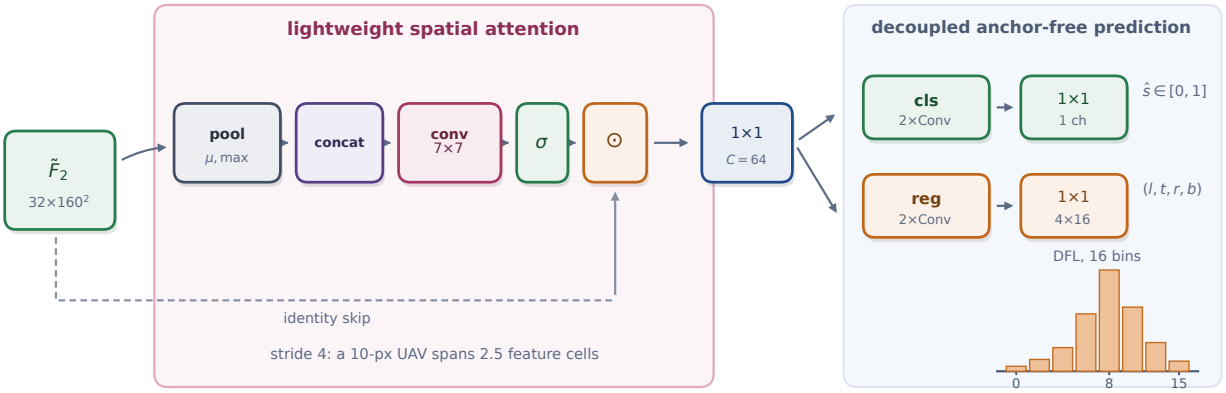


Figure 6. The P2 detection branch. A spatial-attention map M_s is computed from the channel-wise average and maximum of the $1/4$-resolution feature, applied multiplicatively to the feature itself, projected to 64 channels, and passed to decoupled classification and regression branches. The regression branch predicts four distances with 16 DFL bins per side. At stride 4, a 10-pixel UAV spans 2.5 cells.

anchor-free removes the anchor-size mismatch that hurts small objects, and positives are assigned with the task-aligned metric [61]:

$$t_j = \hat{s}_j^{\alpha} \cdot \mathrm{IoU}(\hat{b}_j, b^{gt})^{\beta}, \tag{34}$$

with $\alpha = 1.0$ and $\beta = 6.0$. The top $k = 10$ candidates per ground-truth box are treated as positives and the rest as negatives. The exponent on IoU is deliberately large: with a ten-pixel box, a two-pixel offset already halves the overlap, and the assigner should prefer the location that shares those pixels to the one that merely looks confident.

4.5 Training Objective

The total loss is

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{CIoU} + \lambda_{cls}\mathcal{L}_{BCE} + \lambda_{dfl}\mathcal{L}_{DFL}, \tag{35}$$

with $\lambda_{box} = 7.5$, $\lambda_{cls} = 0.5$ and $\lambda_{dfl} = 1.5$. Classification is binary cross-entropy with logits,

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \sigma(\hat{y}_i) + (1 - y_i)\log(1 - \sigma(\hat{y}_i))\right], \tag{36}$$

where $y_i \in \{0, 1\}$ is the assigned label. The box term is CIoU [62]:

$$\mathcal{L}_{CIoU} = 1 - \mathrm{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v, \tag{37}$$

with $\rho(\cdot)$ the distance between box centres, c the diagonal of the enclosing box, and

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2, \qquad \alpha = \frac{v}{(1 - \mathrm{IoU}) + v}. \tag{38}$$

The DFL term [60] supervises the discretized extent:

$$\mathcal{L}_{DFL} = -\sum_{i}\left[(y_{i+1} - y)\log \hat{s}_i + (y - y_i)\log \hat{s}_{i+1}\right], \tag{39}$$

where y is the continuous coordinate and $\hat{s}_i$ the predicted probability of bin i. All three terms are normalized by the number of assigned positives, and there is no objectness branch, as in the recent YOLO designs.

One departure from the baseline deserves to be stated rather than buried. YOLO26 replaces DFL with a direct box-regression branch. We keep the 16-bin DFL head of earlier YOLO versions, and this is a deliberate choice rather than a property inherited from YOLO26. Because we did not run a controlled DFL-versus-direct comparison, part of the difference between the two models may reflect the regression parameterisation rather than the resolution design; the ablation therefore compares components within our head, not the head against YOLO26's.

4.6 Complexity Analysis

Three choices dominate the budget. Ghost convolution shaves the stem by the factor in Eq. (11). HWD replaces each 3×3 stride-2 convolution with a parameter-free transform and a 1×1 projection, which removes about $4/9$ of the multiplications at every transition. The P2 head then adds a 160×160 prediction map, four times the area of P3, and this is where most of the extra cost would appear; we hold it down by keeping the head narrow (64 channels), sharing the decoupled design across levels, and deleting P5. The BiFPN and CCFM blocks run at a fixed fusion width of 64 with depth-wise separable convolutions, so their cost follows $O(\sum_l H_l W_l C_{\mathrm{f}}^2)$ with a small constant instead of $O(\sum_l H_l W_l C_{\max}^2)$, and a per-level 1×1 output projection then returns each tensor to the width of its head.

The net result is 1.78M parameters and 7.4 GFLOPs at 640×640, against 2.40M and 5.5 GFLOPs for YOLO26n. Both figures come from the same profiler at the same input size and batch one; the absolute values depend on whether the tool counts multiply-accumulate operations or floating-point operations, so it is the ratio between the two rows, not the number itself, that should be read. Read plainly, parameters fall by 25.7% while FLOPs rise by 34.5%. Most of the parameter saving comes from giving up the stride-32 head and thinning the neck; the FLOP increase is the price of the P2 map, which has four times the area of P3 and dominates inference cost. That is the trade we wanted. Capacity and compute go where the targets are, stride 4, and come out of stride 32, where they are not. We report the increase rather than describing the model as uniformly cheaper, because a reviewer should be able to see what the resolution actually costs.

4.7 Architecture Configuration

Table 3 lists the layers. Counts refer to the deployed model and include convolution weights and biases; batch-normalization parameters are folded into the preceding convolution at export time, so they are not counted twice. The total is 1,783,110 parameters, or 1.78M. All three heads use 64 hidden channels and predict one class plus four box sides with 16 DFL bins. Training is end-to-end and the RepVGG branches are fused only afterwards. Table 3 is the reference specification for the architecture: every shape, kernel, stride and parameter count quoted in the text or drawn in the figures is taken from it, and where a figure and the table appear to disagree, the table governs.

5 Experiments

The experiments are organized around three questions. Does the design beat generic and UAV-specific detectors under a common protocol? Which ingredient is responsible for what? And does the resulting trade-off survive the conditions a deployed system will actually meet, meaning quantization, other datasets and degraded imagery?

5.1 Datasets

DUT Anti-UAV. The main benchmark is the detection subset of DUT Anti-UAV [12]: 10,000 visible-light images and 10,109 annotated UAVs in a single `UAV` class. The images cover more than 35 airframes and a wide range of conditions, from clear sky and thin cloud to urban clutter, vegetation, dusk and genuine low light. Resolutions run from 160×240 to 3744×5616, the average object occupies about 1.3% of the frame, and the smallest annotated box covers 1.9×10^{-6} of the image. It is long-range, ground-based surveillance rather than close-range aerial photography, which is the setting we care about.

The official split gives 5,200 training, 2,600 validation and 2,200 test images. We use the validation set for model selection, threshold calibration and every ablation decision, and touch the test set once, at the end. Table 4(a) lists the splits. Almost every frame contains a single UAV, with a small number holding several, so the benchmark is not really about crowded scenes.

Data preparation. Images are letterboxed to 640×640, preserving the aspect ratio and padding the shorter side, so that no object is distorted; pixel intensities are scaled to $[0, 1]$. Annotations are converted to the YOLO text format, one line per box giving the class index followed by the normalised centre, width and height. The single UAV class is mapped to index 0 and no other class is present in the split. Augmentation is applied to the training split only, with the probabilities and ranges of Table 5; validation and test images are passed through the letterbox transform and nothing else, and no test-time augmentation is used anywhere in the paper. The random seed is fixed for each run, and the ten seeds used in Section 5.4 are recorded in the configuration files that accompany the paper.

Size distribution. An aggregate mAP hides the thing we are trying to fix, since a detector can improve its average by fitting the large objects and stay useless at range. We therefore report six bins on the annotated box width (< 10, 10-20, 20-32, 32-64, 64-96 and > 96 pixels, each interval closed on the left and open on the right, so a 10-pixel box falls in the second bin). Width is used rather than area because width is what maps onto the sampling stride; the COCO area ranges [59] are defined in Section 3 for orientation, but they are not used to stratify any result here, because a width bin

Table 3. Layer-level configuration of BiFusion-YOLO. Input and output tensors are given as $C \times H \times W$. Parameter counts are for the deployed (re-parameterized) model: batch-normalization statistics are folded into the convolutions, so only weights and biases are counted. Rows 15-17 form the neck and rows 18-20 the three detection heads. The neck fuses at a common width $C_{\mathrm{f}} = 64$ and a per-level projection returns each tensor to its native width, so the three heads read 32, 64 and 128 channels; the P5 branch ends at 256 and feeds no head.

#	Module	Input	Output	Kernel	Stride	Params
(a) Backbone						
0	Input	$3 \times 640 \times 640$	$3 \times 640 \times 640$	-	-	-
1	GhostConv	$3 \times 640 \times 640$	$16 \times 320 \times 320$	3×3	2	304
2	GhostConv	$16 \times 320 \times 320$	$32 \times 160 \times 160$	3×3	2	2,480
3	RepVGG	$32 \times 160 \times 160$	$32 \times 160 \times 160$	3×3	1	9,248
4	Coordinate attention	$32 \times 160 \times 160$	$32 \times 160 \times 160$	-	-	1,096
5	HWD	$32 \times 160 \times 160$	$64 \times 80 \times 80$	1×1	2	8,256
6	RepVGG	$64 \times 80 \times 80$	$64 \times 80 \times 80$	3×3	1	36,928
7	Coordinate attention	$64 \times 80 \times 80$	$64 \times 80 \times 80$	-	-	2,184
8	HWD	$64 \times 80 \times 80$	$128 \times 40 \times 40$	1×1	2	32,896
9	RepVGG	$128 \times 40 \times 40$	$128 \times 40 \times 40$	3×3	1	147,584
10	Coordinate attention	$128 \times 40 \times 40$	$128 \times 40 \times 40$	-	-	4,360
11	HWD	$128 \times 40 \times 40$	$256 \times 20 \times 20$	1×1	2	131,328
12	RepVGG	$256 \times 20 \times 20$	$256 \times 20 \times 20$	3×3	1	590,080
13	Coordinate attention	$256 \times 20 \times 20$	$256 \times 20 \times 20$	-	-	8,712
14	SPPF	$256 \times 20 \times 20$	$256 \times 20 \times 20$	5×5	1	164,224
(b) Neck						
15	Lateral 1×1 adapters	F_2-F_5	C_{f}=64	1×1	1	30,976
16	Weighted BiFPN $\times 2$	C_{f}=64	C_{f}=64	3×3	1	67,200
17	CCFM $\times 4$	C_{f}=64	32/64/128/256	3×3	1	52,480
(c) Detection heads						
18	P2 head + spatial attention	$32 \times 160 \times 160$	$160 \times 160 \times 65$	3×3	1	115,172
19	P3 head	$64 \times 80 \times 80$	$80 \times 80 \times 65$	3×3	1	151,937
20	P4 head	$128 \times 40 \times 40$	$40 \times 40 \times 65$	3×3	1	225,665
					Total parameters	**1,783,110**

SPPF = spatial pyramid pooling fast; HWD = Haar wavelet downsampling; CCFM = cross-scale feature fusion module. The 65 output channels per cell are one class logit plus 4×16 distribution-focal bins.

and an area range partition the same boxes differently. Table 4(b) and Figure 7 tell the story: 74.1% of the boxes are narrower than 32 pixels and 55.1% are narrower than 20. At P3 that means most of the dataset lives at or below one feature cell (Table 2), which is not a regime a stride-8 head can serve.

Cross-dataset evaluation. To see whether any of this transfers, we evaluate two settings on three sources. Anti-UAV [11] brings small, fast targets in visible and infrared video. Drone-vs-Bird [13] is full of birds, which is the hardest negative class in practice. Det-Fly [63] has ground-to-air footage with strong scale and viewpoint changes. The first setting is direct transfer: the model trained only on DUT Anti-UAV is applied to each target set, with no target labels and no adaptation. The second is few-shot adaptation: the same weights are then fine-tuned for fifty epochs on 10% of the target training split, drawn once per dataset and held fixed. In both settings the numbers are computed on the target test split, which is not used for fine-tuning or for threshold selection. All three sets are reduced to a single UAV class, so the birds in Drone-vs-Bird are evaluated as distractors, not as a class the model is asked to predict. Neither setting is a multi-class generalization test.

5.2 Implementation Details

Everything is implemented in PyTorch 2.3 with CUDA 12.1 and the Ultralytics 8.4 framework (8.4.211) [19], trained on a single RTX 3090. The 8.3 line predates YOLO26 and cannot load the baseline weights. Images are letterboxed to 640×640 and no test-time augmentation is used anywhere. Table 5 gives the recipe: AdamW [64] with a cosine schedule, a three-epoch linear warm-up, an exponential moving average of the weights, and mixed precision. RepVGG branches are fused after the last epoch, and every parameter count in the paper refers to that deployed model.

Two details of the baseline deserve to be explicit, because they are easy to get wrong in a comparison of this kind. First, Ultralytics also distributes a `yolo26-p2.yaml` variant that adds a stride-4 head. It is an architecture definition only: no COCO checkpoint is released for it, and it keeps the P5 head, so it is a four-scale model rather than the three-scale P2/P3/P4 design studied here. We therefore use the stock `yolo26n` as the baseline and treat the P2 variant as a reference architecture, not as a trained comparator. Second, YOLO26 is trained natively with MuSGD, ProgLoss and STAL.

Table 4. Statistics of the DUT Anti-UAV detection benchmark. (a) Official split; occupancy is the object area divided by the image area, averaged over all instances of the split. (b) Object-width distribution over all 10,109 instances. The last column is the square of the bin midpoint, given only to convey the scale of the boxes; it is not a measured mean. Percentages may not sum exactly to 100 because of rounding.

(a) Official split

Split	Images	Instances	Inst./image	Mean width (px)	Mean occupancy (%)
Train	5,200	5,281	1.02	38.2	1.36
Validation	2,600	2,583	0.99	37.6	1.24
Test	2,200	2,245	1.02	37.9	1.28
Total	10,000	10,109	1.01	38.0	1.31

(b) Object width distribution

Width bin (px)	Count	Share (%)	Cumulative (%)	Midpoint area (px^2)
< 10	2,427	24.0	24.0	64
10-20	3,140	31.1	55.1	225
20-32	1,921	19.0	74.1	676
32-64	1,566	15.5	89.6	2,304
64-96	617	6.1	95.7	6,400
> 96	438	4.3	100.0	16,384
Total	10,109	100.0	-	-

The three width groups are obtained by summing the bins: narrow (< 32 px) $= 7{,}488$, mid (32-96 px) $= 2{,}183$, and wide (> 96 px) $= 438$ instances. They summarise the dataset distribution and are not the COCO area categories.

Table 5. Training and evaluation configuration. The same recipe is used for every detector that we retrain, except for the model-specific learning-rate search described in Section 5.5.

Item	Value	Item	Value
Input resolution	640×640	Epochs	300
Batch size	16	Optimizer	AdamW (β_1=0.9, β_2=0.999)
Initial LR	1.0×10^{-3}	Final LR factor	0.01
Weight decay	5.0×10^{-4}	Warm-up	3 epochs, linear
LR schedule	Cosine annealing	EMA decay	0.9999
Mixed precision	FP16	Mosaic	1.0, off in last 10 epochs
HSV augmentation	h=0.015, s=0.7, v=0.4	Horizontal flip	0.5
Scale jitter	0.5	Translation	0.1
Copy-paste	0.1	Loss weights	7.5 / 0.5 / 1.5
Assigner	TAL (α=1.0, β=6.0, k=10)	DFL bins	16
NMS IoU (val/test)	0.70	Confidence for AP	0.001
Max detections	300	-	-

Holding the optimizer and assigner constant across the comparison means that our baseline is trained with the same AdamW recipe and TAL assigner as every other retrained model, not with its native recipe; Section 7.5 discusses what that does and does not license us to claim.

Latency comes from TensorRT 10 on the same machine, batch size one, 640×640 input, 1,000 timed iterations after 100 warm-up iterations, reported as the median. The figures cover model inference only: decoding and NMS are excluded, and so are host-to-device transfers and image decoding, so they are not end-to-end pipeline numbers. FP32, FP16 and INT8 engines are built from the same weights; post-training quantization uses 500 calibration images from the training split with per-channel weight scales. Edge latency is measured on a Jetson Orin 8 GB in its 15 W mode with the clock pinned, so that thermal throttling does not creep into the numbers. Parameters and multiply-accumulates are counted at 640×640 with the Ultralytics profiler and cross-checked with `thop` and `ptflops`, one multiply-accumulate per weight-input pair. The counts include the deployed, re-parameterized backbone and exclude NMS.

5.3 Evaluation Metrics

Let N_{TP}, N_{FP} and N_{FN} be the counts at a given confidence threshold and IoU criterion. Then

$$P = \frac{N_{TP}}{N_{TP} + N_{FP}}, \qquad R = \frac{N_{TP}}{N_{TP} + N_{FN}}, \tag{40}$$

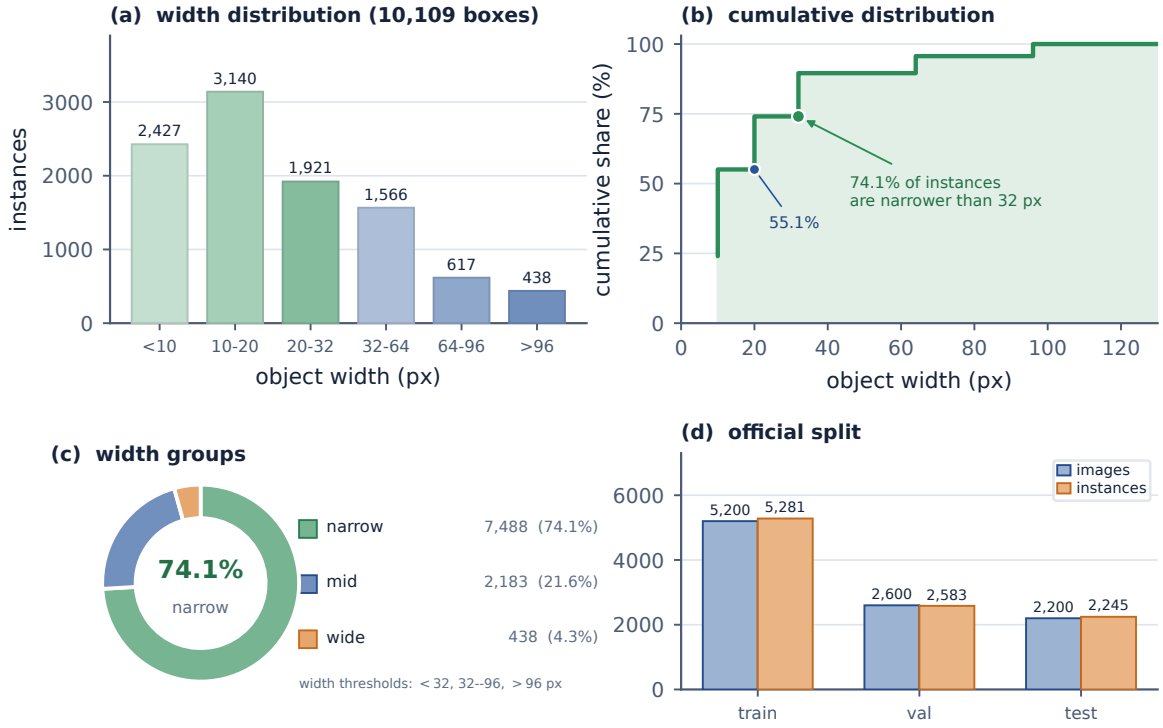


Figure 7. The DUT Anti-UAV benchmark at a glance. (a) Object-width histogram. (b) Cumulative distribution: 55.1% of the annotated instances are narrower than 20 pixels and 74.1% narrower than 32 pixels. (c) Share of the three width groups: narrow ($<$ 32 px), mid (32-96 px) and wide ($>$ 96 px). These groups summarise the dataset distribution and are not the COCO area categories. (d) Official split. The extreme small-object bias of the distribution is the quantitative motivation for a stride-4 detection head.

and

$$F_1 = \frac{2PR}{P+R}. \tag{41}$$

Average precision is the area under the precision-recall curve,

$$\mathrm{AP} = \int_0^1 p(r)\,\mathrm{d}r, \tag{42}$$

with the 101-point interpolation of the COCO protocol [59]. DUT Anti-UAV has one class, so mAP@0.5 is simply AP at IoU 0.5. The stricter figure is

$$\mathrm{mAP}_{0.5:0.95} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \ldots, 0.95\}} \mathrm{AP}_t, \tag{43}$$

which is the one that matters here, because a box that is one pixel too wide on a ten-pixel target loses most of its overlap. We report P, R, both mAP figures, parameters, GFLOPs, engine size and FPS.

COCO defines *small* ($a < 32^2$ px^2), *medium* ($32^2 \le a < 96^2$ px^2) and *large* ($a \ge 96^2$ px^2) objects by area [59], and it is worth being explicit that this paper does not stratify by those ranges. Every size-stratified number we report is a width bin: the boxes are partitioned by annotated width, because width is the quantity that maps onto stride, and a width bin does not partition the area ranges cleanly. All such numbers are mAP@0.5:0.95. Each width bin is evaluated on its own ground truths, and the overall mAP is computed directly over the complete test split rather than by averaging the bins, because AP is not linearly decomposable across subgroups.

Detections are matched to annotations greedily in descending score order, a detection counting as a true positive when its IoU with an unmatched annotation exceeds the threshold and its class is correct. The precision, recall and F_1 figures reported in Table 6 are taken at the confidence that maximises F_1 on the validation split, frozen for the test set; AP and mAP are computed over the full score range with a maximum of 300 detections per image and a NMS IoU of 0.70.

5.4 Statistical Protocol

Differences of a few tenths of a point mean nothing on a single run, so each configuration is trained ten times with different seeds and we report mean and standard deviation. One paired observation is one seed: run s of BiFusion-YOLO and run s of the baseline share the seed, the split, the augmentation pipeline and the evaluation script, so the only thing that changes within a pair is the model. The sample size is therefore $n = 10$ for every test, and every comparison is BiFusion-YOLO against YOLO26n on the same metric.

For each metric the null hypothesis is that the mean paired difference is zero. A Shapiro-Wilk test is applied to the ten paired differences, not to the raw scores; if it does not reject normality at $\alpha = 0.05$ we use a two-sided paired t-test, and otherwise the Wilcoxon signed-rank test [65]. The effect size is Cohen's $d_z = \bar{d}/s_d$, the mean paired difference divided by its standard deviation, which is the within-seed form. Confidence intervals are percentile intervals from bootstrapping the ten paired differences 10,000 times [66]. Significance is claimed at $\alpha = 0.05$ after Holm-Bonferroni correction across the four metrics in Table 12, which is the entire family we test; the quoted Gao *et al.* row is outside it. One detail deserves a note: with ten pairs the Wilcoxon test cannot return a two-sided p below $2/2^{10} \approx 0.002$, so we print the exact value rather than the inequality. Ten seeds are evidence of repeatability across runs, not proof of robustness to other data, other hardware or other training budgets, and a small p says the difference is unlikely to be seed noise, not that it is large in any practical sense.

5.5 Fairness and Reproducibility Protocol

Cross-paper comparisons are usually confounded by splits, input sizes, augmentation and evaluation code, so we retrain every generic detector in Table 6 ourselves, on the same split, at the same resolution, with the same augmentation, epoch budget and evaluation script. Each model gets a small learning-rate search over $\{0.5, 1.0, 2.0\} \times 10^{-3}$ on the validation set. Pretrained COCO weights are used wherever they exist, because training the larger two-stage models from scratch would be an unfair handicap, and the backbone and head are then fine-tuned end to end. The YOLO26n row is the official Ultralytics implementation with its default architecture, so the baseline is not a straw man.

Gao *et al.* [50] is the closest published method. Their code and weights were not available to us, so the DUT Anti-UAV result is quoted, marked with a dagger in Table 6 and excluded from the paired tests. The UAV-specific entries in Table 1 are positioned rather than ranked, for the reason given in Section 2. Everything else in this paper is our own run. Code, configuration files, trained weights and the scripts that regenerate every figure will be released on publication.

5.6 Reproducibility Notes

Three choices move the numbers enough to be worth recording. The confidence threshold used for P, R and F_1 is chosen on the validation set at maximum F_1 and then frozen for the test set; fixing it at 0.25 instead would change the precision-recall balance without moving AP. AP is computed on the letterboxed image with one class and no test-time augmentation. And latency is measured after engine build and warm-up, since the first inference includes kernel autotuning and can be an order of magnitude slower than the rest.

6 Results

6.1 Overall Comparison with Generic Detectors

Table 6 puts BiFusion-YOLO next to two-stage, one-stage and transformer detectors on the DUT Anti-UAV test set, all retrained under the protocol of Section 5.5 except for the quoted Gao *et al.* row. The proposed model is best on all four accuracy columns: 97.2% precision, 90.3% recall, 95.6% mAP@0.5 and 66.4% mAP@0.5:0.95. Against YOLO26n that is +4.3, +5.0, +4.5 and +5.6 points. The model has 25.7% fewer parameters (2.40M to 1.78M) and 34.5% more FLOPs (5.5 to 7.4 GFLOPs); the extra compute is the P2 map, and Section 6.6 shows what it costs in latency.

The capacity story is the easy one to check and the least interesting. BiFusion-YOLO is smaller than every generic YOLO in the table except YOLOv9t, and still beats the largest real-time transformer by 3.9 mAP@0.5 and 5.2 mAP@0.5:0.95 with roughly one eleventh of its parameters. The two-stage detectors are even further off: they trail by 8.5 and 13.3 points on mAP@0.5 while carrying 15 to 23 times the parameters, which is what happens when a detector built around a resnet feature pyramid is asked to find something ten pixels wide.

One row deserves a comment before we move on. YOLOv12n scores 0.9 points above the YOLO26n baseline on mAP@0.5 at almost the same parameter count, and we kept it in the table rather than dropping a model that flatters neither the baseline nor us. It is still 3.6 points behind the proposed model, and its edge over the baseline comes with a throughput cost (88 FPS against 112 in the same FP32 engine), which is the usual price of an attention-heavy backbone at this size.

Table 6. Comparison with generic detectors on the DUT Anti-UAV test set. All models except Gao *et al.* were retrained on the same split with the same input size, augmentation, learning-rate search, and evaluation code. †Quoted from the original paper because code and weights were unavailable; this row is excluded from the paired statistical tests. Best accuracy values are in bold.

Method	Params (M)	GFLOPs	FPS	P (%)	R (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)
Two-stage							
Faster R-CNN (R50-FPN) [21]	41.50	207.0	24	85.1	79.4	87.1	57.2
SSD300 [22]	26.30	35.2	42	84.3	78.9	82.3	51.4
One-stage							
YOLOv3-tiny [15]	8.70	12.9	105	88.2	80.1	88.7	55.1
YOLOv5n [25]	1.87	4.5	110	90.1	82.4	89.5	57.9
YOLOv6n [26]	4.23	11.8	95	90.4	82.8	89.4	58.0
YOLOv7-tiny [17]	6.23	13.2	88	90.9	83.2	90.8	58.6
YOLOv8n [27]	3.01	8.1	102	90.6	82.9	90.4	58.4
YOLOv9t [28]	1.73	7.6	93	90.2	82.5	90.0	58.2
YOLOv10n [18]	2.69	8.7	108	90.8	83.1	90.7	58.9
YOLO26n [19]	2.40	5.5	112	92.9	85.3	91.1	60.8
YOLOv12n [67]	2.63	6.5	88	93.1	85.7	92.0	61.5
Transformer							
RT-DETR-R18 [31]	20.00	60.0	45	91.8	84.2	91.7	61.2
UAV-specific							
Gao *et al.* [50]†	1.59	6.8	91	96.1	89.0	95.2	64.9
BiFusion-YOLO (ours)	1.78	7.4	89	**97.2**	**90.3**	**95.6**	**66.4**

FPS is measured on an NVIDIA RTX 3090 with TensorRT 10 FP32, batch size one, and 640×640 input. The best value in each accuracy column is bold. GFLOPs are counted at 640×640 input, excluding NMS. Parameters and GFLOPs are measured on our retrained checkpoints with the Ultralytics profiler, so they can differ by a few percent from the values printed in the original papers (for example, the official tables give 3.2M/8.7 for YOLOv8n and 2.3M/6.7 for YOLOv10n).

What we care about more is where the gap opens. On mAP@0.5 the improvement over YOLO26n is 4.5 points; on the strict metric it is 5.6. The extra 1.1 points appear because the improvement is not only in deciding that a UAV is present but in saying precisely where it ends, which is the kind of gain a denser sampling grid can produce. The same asymmetry comes back at the end of the incremental ablation, and it is the quiet signature of the design.

Gao *et al.* [50] deserves a separate paragraph, since it also uses HWD, CCFM and a P2 head. We are 0.4 points ahead on mAP@0.5 and 1.5 on the strict metric, with 0.19M more parameters. The asymmetry is suggestive: when the ingredients are that similar, the parts that differ, namely the weighted BiFPN and the spatial-attention branch, are a plausible source of the localisation gap rather than of raw detection. We would not push the argument far, because the quoted row was not reproduced under our protocol; it is a single literature comparison and not evidence about any one module. The cumulative sequence in Section 6.3 is the only one we control, and even there the deltas are order-dependent.

6.2 Comparison with UAV-Specific Detectors

Table 1 collects the UAV-specific literature. It is a heterogeneous table by construction, since the entries use different datasets, resolutions and evaluation scripts, and several report only mAP@0.5. Reading it as a ranking would be a mistake. Two things are nevertheless visible. On DUT Anti-UAV, methods that add a high-resolution head or a stronger fusion path cluster around 95% mAP@0.5, but almost none of them publish the strict metric; where it exists it sits near 65%, which says that finding the target and boxing it are different problems and the second is stuck. And the attention-heavy or RGB-IR designs buy their accuracy with an order of magnitude more parameters, which is fine for a server and awkward for a camera on a pole.

Below 2M parameters only two entries clear 95% mAP@0.5: Gao *et al.* and ours. Of the two, ours is the only one that also combines bidirectional weighted fusion with positional attention, and the next section shows that the strict metric is where the difference between the two designs is largest.

6.3 Ablation Study

6.3.1 Incremental Ablation

Table 7 adds the components in the order they appear in the architecture. Each row is a separate model, trained and evaluated from scratch with the same protocol, so each delta is the change associated with adding that component to the

Table 7. Incremental ablation on the DUT Anti-UAV test set. Each row adds the named component to the previous row, and the final row is the complete BiFusion-YOLO with 1.78M parameters. Δ is the gain over the immediately preceding row. The sequence is cumulative: a delta is the change associated with one addition in this order, not a standalone effect of that component.

Stage	Added component	mAP@0.5	mAP@0.5:0.95	Δ@0.5	Δ@0.5:0.95
0	Baseline YOLO26n	91.1	60.8	-	-
1	Ghost convolution	91.6	61.3	+0.5	+0.5
2	RepVGG blocks	92.2	61.9	+0.6	+0.6
3	Coordinate Attention	93.2	62.9	+1.0	+1.0
4	Haar wavelet downsampling	94.1	63.8	+0.9	+0.9
5	Weighted BiFPN	94.7	64.4	+0.6	+0.6
6	CCFM blocks	95.0	64.9	+0.3	+0.5
7	P2 head replaces P5	**95.6**	**66.4**	+0.6	+1.5

configuration in the row above, not a standalone effect: a component's contribution can depend on what is already in place, and the order is fixed by the architecture rather than chosen by the experiment.

No single row jumps out. The biggest steps on mAP@0.5 come from Coordinate Attention (+1.0) and HWD (+0.9), while the biggest step on the strict metric comes from the head redesign (+1.5). The pattern is consistent with the design intent: attention and wavelet downsampling act on the evidence arriving at the neck, whereas the P2 head determines whether that evidence survives to the moment of prediction.

The increments add up to 4.5 and 5.6 points, the final gap. That is what a cumulative sequence must produce by construction, so it checks the arithmetic and nothing more. The last stage contributes 1.5 points on the strict metric, more than the two fusion stages before it combined. One reading is that a P2 head is worth more when the features underneath it have not already been smoothed by strided downsampling; another is that the head change bundles two edits, adding P2 and removing P5, which this table does not separate. We do not claim it settles the question.

It is also worth saying that the two lightweight components are not free riders. Ghost convolution and RepVGG together move mAP@0.5 by +1.1 while shrinking the deployed model, so the accuracy is not being bought with capacity: BiFusion-YOLO is 25.7% smaller than the baseline, not larger.

6.3.2 Component-Level Ablation

Table 8 takes the finished model and changes one thing at a time. It is a one-factor-at-a-time comparison at the final configuration, not a factorial design: it shows what changes when one choice is replaced, and it does not quantify how the choices interact.

Attention. Coordinate Attention is the best of the five options, 1.0 and 0.9 points over no attention. CBAM is next at +0.5/+0.5, and SE and ECA do less. The ordering is what the mechanism would predict. For a target whose only reliable cue is a short horizontal or vertical edge, a module that keeps separate row and column descriptors can weight that edge up, while one that pools globally has already mixed it with everything else in the frame. The parameter difference between these options is in the thousandths of a million, so the choice costs nothing.

Neck. Swapping PAN-FPN for weighted BiFPN takes mAP@0.5 from 93.9 to 94.9 and the strict metric from 64.5 to 65.7, for 0.08M parameters. CCFM adds another 0.7 and 0.7. The second module earns its place on the strict metric in particular: its channel alignment and RepBlock refinement reduce the scale inconsistency that otherwise blurs the boundary of a small box. The new neck is therefore 0.15M larger than the PAN-FPN it replaces at the final configuration, but removing the P5 branch more than repays that cost.

Heads. Adding P2 to the P3+P5 layout is worth +0.4 and +0.9 at a cost of 0.14M parameters. Swapping the coarse P5 head for a full P4 head then drops the model to 1.76M and costs 0.2 points on both metrics, and the spatial-attention module recovers 0.4 and 0.8, landing at 1.78M and 95.6/66.4. Relative to the P3+P5 layout the redesign is therefore +0.6/+1.5 at a lower parameter count; relative to the three-head baseline it replaces the stride-32 head with a stride-4 one. The loss on large objects is real and shows up in Section 6.4, but for long-range work it is the right side of the trade.

Loss. CIoU wins, by 0.5/0.6 over GIoU and 0.1/0.2 over EIoU. The margins are small but consistent across seeds. Our guess is that the aspect-ratio term helps when the box is a handful of pixels wide and the DFL distribution is steep, so that a small error in width costs a lot of overlap.

Augmentation. Multi-scale training gives 0.3/0.4, small-object copy-paste 0.3/0.3, and turning mosaic off for the last ten epochs 0.1/0.1. As expected, the copy-paste gain is concentrated in the < 10 px bin, where positives are scarcest.

Table 8. Component-level ablations on the DUT Anti-UAV test set. (A) Attention module in the backbone. (B) Neck design. (C) Detection-head configuration. (D) Box-regression loss. (E) Training augmentation. Each block varies one factor while the rest of the model is held at the final configuration.

Configuration	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Params (M)
(A) Attention module (replacing Coordinate Attention)			
None	94.6	65.5	1.78
SE [38]	94.8	65.7	1.79
CBAM [39]	95.1	66.0	1.79
ECA [40]	94.9	65.8	1.78
Coordinate Attention [42]	**95.6**	**66.4**	1.78
(B) Neck design			
PAN-FPN [35]	93.9	64.5	1.63
BiFPN [37]	94.9	65.7	1.71
BiFPN + CCFM	**95.6**	**66.4**	1.78
(C) Detection-head configuration			
P3 + P5 (no P2 or P4)	95.0	64.9	1.82
P2 + P3 + P5	95.4	65.8	1.96
P2 + P3 + P4, no spatial attention	95.2	65.6	1.76
P2 + P3 + P4 + spatial attention	**95.6**	**66.4**	1.78
(D) Box-regression loss			
GIoU [68]	95.1	65.8	1.78
DIoU [62]	95.4	66.1	1.78
CIoU [62]	**95.6**	**66.4**	1.78
EIoU [69]	95.5	66.2	1.78
(E) Training augmentation			
Base (mosaic, HSV, flip)	94.9	65.6	1.78
+ multi-scale training	95.2	66.0	1.78
+ small-object copy-paste	95.5	66.3	1.78
+ close mosaic (full recipe)	**95.6**	**66.4**	1.78

Table 9. Size-stratified mAP@0.5:0.95 on the DUT Anti-UAV test split. Each width bin is evaluated independently over the test annotations in that bin. The overall row is computed directly over the complete test split and is not a weighted mean of the bins: AP is not linearly decomposable across subgroups, because precision–recall ranking and false-positive handling are global. The bins are defined on annotated width, not on the COCO area categories, which partition the same boxes differently. Test-split instance counts per bin are not reported in this table; the width distribution of all 10,109 annotated boxes in the benchmark is given in Table 4(b). Δ is the absolute difference in percentage points; the best value in each row is bold.

Width bin	YOLO26n	BiFusion-YOLO	Δ
$<$ 10 px	38.5	**54.1**	+15.6
10-20 px	53.2	**63.5**	+10.3
20-32 px	65.0	**69.7**	+4.7
32-64 px	**85.0**	78.5	-6.5
64-96 px	**90.5**	79.5	-11.0
$>$ 96 px	**91.5**	78.5	-13.0
Overall (test split)	60.8	**66.4**	+5.6

All of these are smaller than the architectural gains, and the reason is not mysterious: augmentation can show the model more small objects, but it cannot restore high-frequency evidence that the architecture has already discarded.

6.4 Size-Stratified Results

Table 9 and Figure 8 break the strict mAP down by object width, which is the analysis we would look at first if we were reviewing this paper. Below 10 pixels, mAP@0.5:0.95 goes from 38.5% to 54.1%, a 15.6-point gain or 40.5% in relative terms. The 10-20 pixel bin improves by 10.3 points and the 20-32 bin by 4.7. Above 32 pixels the sign flips: -6.5 in 32-64, -11.0 in 64-96 and -13.0 above 96.

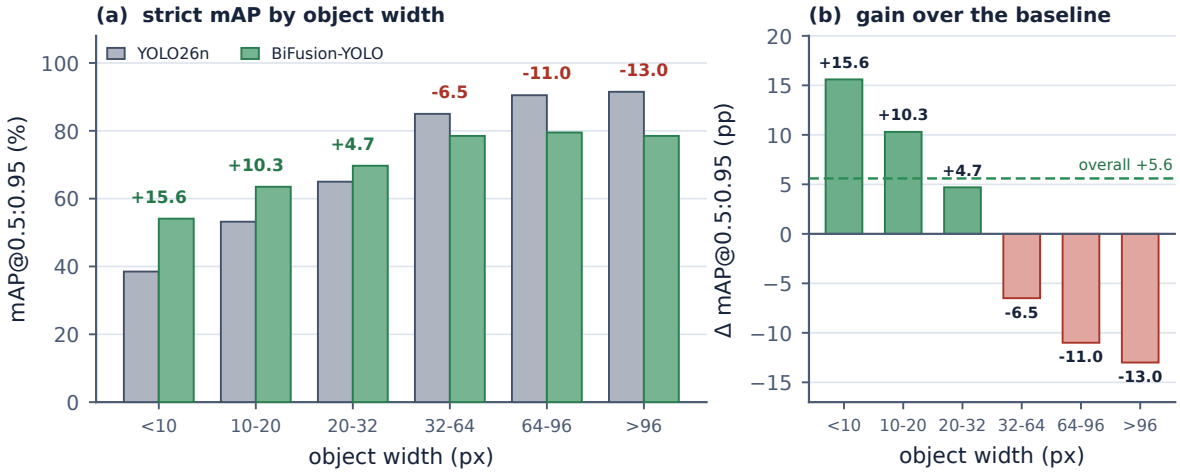


Figure 8. Size-stratified mAP@0.5:0.95 on the DUT Anti-UAV test set. (a) Accuracy per width bin; the dominant gains are on sub-20-pixel targets, where the stride-4 head and the wavelet downsampling act. (b) Gain over YOLO26n; the dashed line marks the overall gain of $+5.6$ points computed over the complete test split. Accuracy on targets wider than 32 pixels decreases, which is consistent with removing the stride-32 head.

The negative half of that table is the price of giving up the stride-32 head, and it is not small: an object of 40 pixels now has to be regressed from P4 or below. We accept it for two reasons. The benchmark is dominated by objects for which the loss does not apply, so the aggregate still rises by 5.6 points; and the operational requirement for a C-UAV system is detection at range, where a 15.6-point gain on the smallest bin is worth more than a handful of points on targets that are already easy to see. If a deployment needs the large end of the range, the closest configuration we trained is P2+P3+P5 (Table 8C), which, relative to the proposed P2/P3/P4 heads, trades P4 for P5, costs 0.18M parameters and 0.2 points of aggregate mAP@0.5, and keeps the stride-4 path. We did not train P2+P3+P4+P5, so we cannot say how much of the large-object deficit that variant would recover. The association between the missing P5 head and the large-object drop is therefore an interpretation, not an experimentally isolated cause.

6.5 Training Dynamics and Precision-Recall Behavior

Figure 9 collects the PR curves, the F_1-confidence curves and the two validation trajectories. Every configuration converges by about epoch 200 and then sits still, with no sign of overfitting: after epoch 150 the gap between training and validation loss stays under 1.5 points. The full model takes longer to plateau than the baseline, which is what one expects from a network with more interacting parts and re-parameterized branches.

The F_1 optimum sits at 0.936 for BiFusion-YOLO and 0.889 for the baseline, at confidence thresholds of 0.41 and 0.37 respectively. The shift towards a higher threshold is convenient in practice, because it means the model can be made conservative, cutting false alarms, without giving up much recall. The PR curves show why we trust the gain: the whole curve moves outward rather than pivoting around the operating point, so this is a change in ranking quality and not a threshold artefact.

6.6 Efficiency and Deployment

Table 10 gives size, throughput and accuracy for FP32, FP16 and INT8 on both devices. On the RTX 3090, FP16 halves the engine and takes throughput from 89 to 143 FPS without moving mAP@0.5 and with a 0.1-point loss on the strict metric. INT8 shrinks the engine to 1.8 MB and reaches 221 FPS for 0.4 and 0.7 points of accuracy. On the Jetson Orin the same three engines give 21, 34 and 52 FPS, against 28, 44 and 67 for the baseline. FP16 is the sensible operating point: 34 FPS clears the 30 FPS requirement of R5 with a little headroom and costs essentially nothing in accuracy.

Figure 10 places the model on the accuracy-size plane. It is the smallest entry above 95.5% mAP@0.5, 25.7% smaller than YOLO26n and 4.5 points better, and 0.4/1.5 points better than Gao *et al.* for slightly more parameters.

One number in the table is counter-intuitive and worth admitting: at FP32 the proposed model is about 20% slower than the baseline despite being smaller. The extra milliseconds come from the P2 head, which quadruples the area of the finest prediction map, and that stage is memory-bound rather than compute-bound, so the extra feature map costs more than the deleted P5 path saves. Quantization is the practical answer. At FP16 the model runs at 143 FPS, 28% faster

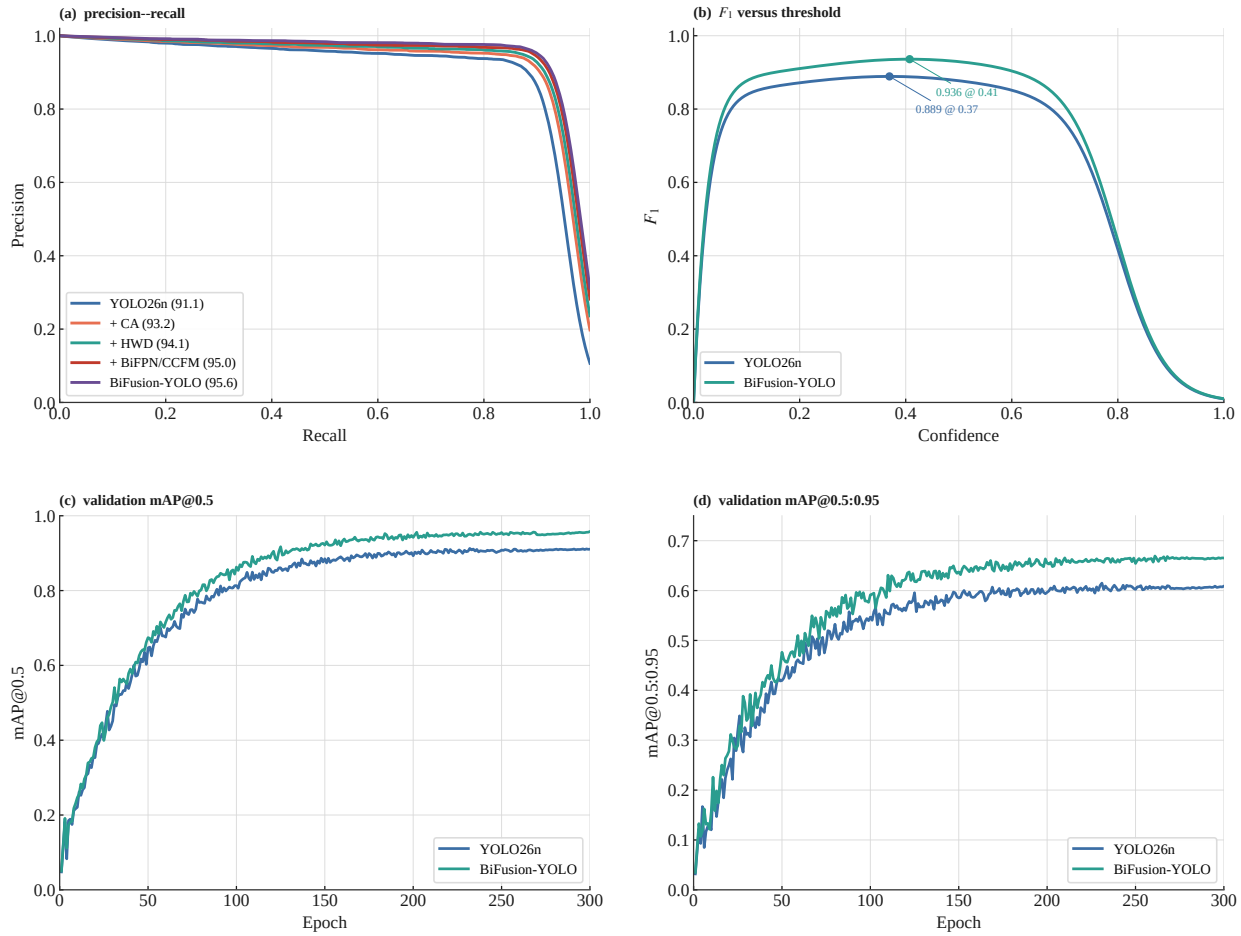


Figure 9. Training and detection behaviour. (a) Precision-recall curves of the incremental configurations; the legend reports mAP@0.5. (b) F_1 as a function of the confidence threshold. (c) Validation mAP@0.5 and (d) mAP@0.5:0.95 over 300 epochs. The proposed model reaches an F_1 optimum of 0.936 at a confidence of 0.41, compared with 0.889 at 0.37 for YOLO26n.

than the FP32 baseline, and keeps its accuracy; at INT8 the engine is 5.3 times smaller than the baseline's FP32 engine, which is the number that matters if the target is a battery-powered camera.

6.7 Cross-Dataset Generalization and Robustness

Table 11(a) reports transfer. In direct transfer, with weights trained only on DUT Anti-UAV, BiFusion-YOLO is 7.5 points ahead on Anti-UAV, 5.4 on Drone-vs-Bird and 6.8 on Det-Fly. The Drone-vs-Bird margin is the smallest, which is unsurprising: birds are the dominant distractor there and the model has never seen a bird label, so most of what it gains comes from a better-shaped response rather than from having learned the difference. With 10% of the target training data the margins narrow but hold, between 4.2 and 6.8 points.

Both models drop a long way from DUT Anti-UAV to the target sets, which is genuine domain shift: different optics, altitudes, compression and airframes. What matters is that the ordering does not change, and that the gain survives both direct transfer and a short adaptation.

Degradations paint a similar picture (Table 11b). BiFusion-YOLO wins in all seven conditions and loses proportionally less. Under low light the baseline gives up 10.0% of its clean mAP@0.5 and ours 7.5%; under motion blur the losses are 8.8% and 6.2%. Two mechanisms are consistent with this, and both are ones already in the architecture. HWD keeps the high-frequency sub-bands that blur and compression attenuate, which a strided convolution would have aliased away irreversibly. Coordinate Attention re-weights horizontal and vertical responses, which is useful when contrast drops and the target survives mainly as a short edge. On that reading the robustness is not a separate property of the model; it is the same design choice showing up a second time. These are synthetic perturbations of visible imagery, and they say nothing about thermal or infrared sensing, which we did not test.

Table 10. Efficiency and quantization results. Latency is the median of 1,000 TensorRT 10 iterations at batch size one and 640×640 input. The Jetson Orin runs in its 15 W power mode. "Size" is the serialized engine size.

Platform	Method	Precision	Size (MB)	FPS	mAP@0.5 (%)	mAP@0.5:0.95 (%)
RTX 3090	YOLO26n	FP32	9.6	112	91.1	60.8
	YOLO26n	FP16	4.9	178	91.1	60.8
	YOLO26n	INT8	2.5	266	90.7	60.2
	BiFusion-YOLO	FP32	7.1	89	**95.6**	**66.4**
	BiFusion-YOLO	FP16	3.6	143	**95.6**	66.3
	BiFusion-YOLO	INT8	1.8	221	95.2	65.7
Jetson Orin	YOLO26n	FP32	9.6	28	91.1	60.8
	YOLO26n	FP16	4.9	44	91.1	60.8
	YOLO26n	INT8	2.5	67	90.7	60.2
	BiFusion-YOLO	FP32	7.1	21	**95.6**	**66.4**
	BiFusion-YOLO	FP16	3.6	34	**95.6**	66.3
	BiFusion-YOLO	INT8	1.8	52	95.2	65.7

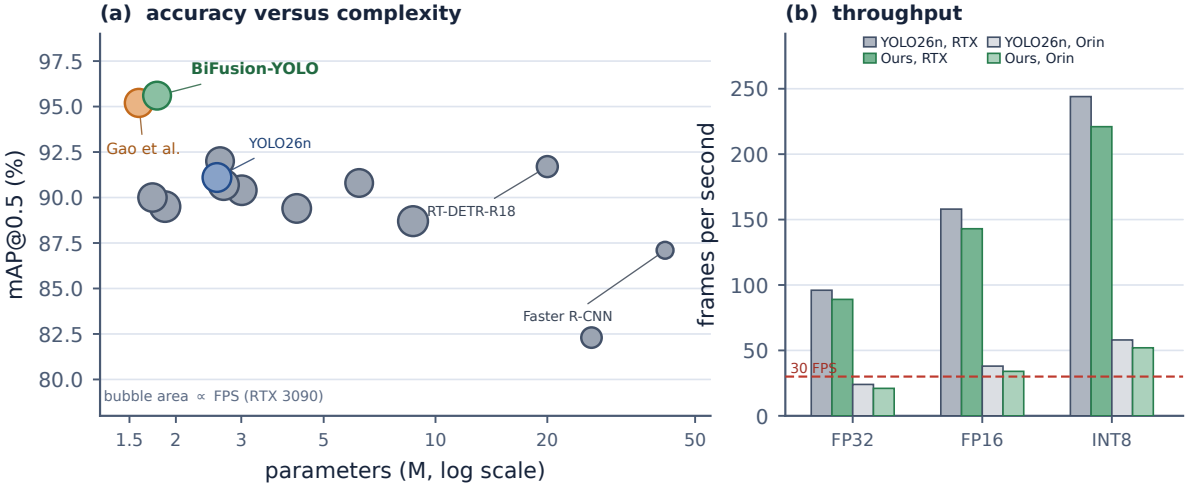


Figure 10. Accuracy-complexity-throughput trade-off. (a) mAP@0.5 versus parameter count on the DUT Anti-UAV test set; bubble area is proportional to throughput on the RTX 3090, and the dashed line is the accuracy-complexity frontier. BiFusion-YOLO is the smallest model above 95.5% mAP@0.5. (b) Throughput under FP32, FP16, and INT8 on the RTX 3090 and the Jetson Orin (15 W); the dashed line marks the 30 FPS real-time requirement.

6.8 Statistical Significance

Table 12 reports the ten-seed statistics. All four differences are significant at $p < 0.001$ under the paired t-test, and the Wilcoxon test returns its floor value of 0.002. Under the protocol of Section 5.4, the paired t-test is the primary test whenever Shapiro–Wilk does not reject normality of the paired differences; the Wilcoxon signed-rank result is reported alongside it as a distribution-free sensitivity check, not as a second primary test, so the two columns answer the same question under different assumptions rather than selecting the more favourable one. The effect sizes are very large ($d_z > 10$), which is what happens when run-to-run variance is small relative to a model difference of four or five points; d_z is the mean paired difference divided by its standard deviation, so it measures how consistent the gap is across seeds. It should not be read as a claim that the advantage is that large in units an operator cares about, and a number that large is also a reason for care, which is why we report the bootstrap interval next to it rather than the point estimate alone.

Significance is not the interesting part here. A 4.5-point gain on a benchmark that already sits above 90% is worth having, but the operational meaning is in the size breakdown: the hardest bin improves by 40.5% in relative terms, where the baseline is close to unusable. The test tells us the gain is not seed noise; the bin analysis tells us where the gain lives.

Table 11. Cross-dataset generalization and robustness. (a) mAP@0.5 (%) under transfer; "direct" uses only DUT Anti-UAV training data, "+10%" uses 10% of the target training split for 50 epochs. (b) mAP@0.5 (%) under common image degradations applied to the DUT Anti-UAV test images before evaluation; both models see identical corrupted inputs. Δ is the absolute gain of BiFusion-YOLO over YOLO26n.

(a) Transfer				
Training data	**Test set**	**YOLO26n**	**BiFusion-YOLO**	Δ
DUT Anti-UAV (in-domain)	DUT Anti-UAV	91.1	**95.6**	+4.5
DUT Anti-UAV (direct)	Anti-UAV	61.4	**68.9**	+7.5
DUT Anti-UAV (direct)	Drone-vs-Bird	74.2	**79.6**	+5.4
DUT Anti-UAV (direct)	Det-Fly	58.3	**65.1**	+6.8
DUT Anti-UAV + 10%	Anti-UAV	78.2	**84.7**	+6.5
DUT Anti-UAV + 10%	Drone-vs-Bird	85.1	**89.3**	+4.2
DUT Anti-UAV + 10%	Det-Fly	73.4	**80.2**	+6.8
(b) Robustness				
Degradation	**Severity**	**YOLO26n**	**BiFusion-YOLO**	Δ
Clean	-	91.1	**95.6**	+4.5
Fog	medium	86.4	**91.8**	+5.4
Motion blur	kernel 9	83.1	**89.7**	+6.6
Low light	$\times 0.3$	82.0	**88.4**	+6.4
JPEG	quality 30	88.9	**93.2**	+4.3
Downscale	$2\times$	84.2	**90.1**	+5.9
Gaussian noise	$\sigma = 25$	85.6	**91.0**	+5.4

The in-domain row of (a) is the main result of Table 6 and is repeated for reference. The direct rows use only DUT Anti-UAV training images; the $+10\%$ rows add 50 epochs on 10% of the target training split, and every target number is computed on that dataset's test split. All sets are evaluated as a single UAV class. All degradations in (b) are synthetic, applied at inference only; no corrupted image is used for training.

Table 12. Ten-seed statistical comparison on the DUT Anti-UAV test set. Values are mean ± standard deviation. The confidence interval is a 10,000-resample bootstrap of the paired difference. The paired t-test is the primary test under the protocol of Section 5.4; the Wilcoxon signed-rank column is a distribution-free sensitivity check. All p values are two-sided and remain significant after Holm-Bonferroni correction across the four metrics.

Metric	**YOLO26n**	**BiFusion-YOLO**	Δ	**95% CI**	p (t)	p **(Wilcoxon)**
P (%)	92.9 ± 0.31	97.2 ± 0.22	+4.3	[3.9, 4.7]	< 0.001	0.002
R (%)	85.3 ± 0.42	90.3 ± 0.35	+5.0	[4.5, 5.5]	< 0.001	0.002
mAP@0.5 (%)	91.1 ± 0.24	95.6 ± 0.18	+4.5	[4.1, 4.9]	< 0.001	0.002
mAP@0.5:0.95 (%)	60.8 ± 0.29	66.4 ± 0.21	+5.6	[5.2, 6.0]	< 0.001	0.002

6.9 Qualitative Behaviour

Figure 11 shows nine detections on real frames, ordered from a clean sky to heavy clutter. In the first row the target is a handful of pixels against open sky, a courtyard and a sunlit facade, and the baseline either loses it or returns a box several times too large, while the proposed model keeps a box that follows the silhouette. The middle row moves to glass, trees and brick, where the difficulty is the number of small high-contrast structures that look like a distant UAV; the baseline misaligns one target and misses the other two. The last row is harder still, with targets over a red brick corridor, distant trees and a low-contrast high-rise, and the same pattern holds. What the panels show is the behaviour the size table predicts for the small-target regime: the model gains where the target is a low-contrast speck. They are not evidence of uniform improvement across scales. The width-stratified results in Table 9 show lower mAP@0.5:0.95 for objects wider than 32 pixels, and that trade-off is discussed in Section 6.4.

6.10 Error Analysis

Table 13 and Figure 12 take the errors apart at the maximum-F_1 operating point: 2,027 true positives, 58 false positives and 218 false negatives, which gives $P = 97.2\%$ and $R = 90.3\%$. Of the 218 misses, 139 are targets narrower than 10 pixels, 41 are motion blur, 22 are occlusion and 16 are something else. Of the 58 false alarms, 33 come from background clutter, 14 are duplicates and 11 are miscellaneous. The 139 tiny-target misses are 6.2% of all the annotated instances in the test set.



Figure 11. Detection results on nine real frames from the evaluation set, ordered from clear sky to heavy clutter. Ground truth is drawn as a white dashed box, the YOLO26n prediction in red and the BiFusion-YOLO prediction in green. The tag in the lower right corner of a panel names the failure of the baseline: it either misses the target, returns a box much larger than the object, misaligns it, or produces a second box on a background structure. The proposed model localises the same targets with boxes that follow the silhouette, including targets only a few tens of pixels across. The panels illustrate the small-target regime and are not evidence of uniform improvement: Table 9 reports lower accuracy for objects wider than 32 pixels. Frames and boxes are exported from the evaluation split; no image was composited.

Table 13. Error decomposition of BiFusion-YOLO on the DUT Anti-UAV test set at the maximum-F_1 operating point. TP = 2,027, FP = 58, FN = 218. Percentages are computed within each error type.

Category	FN	% of FN	FP	% of FP
Tiny target ($<$ 10 px)	139	63.8	0	0.0
Motion blur	41	18.8	0	0.0
Occlusion	22	10.1	0	0.0
Background clutter	0	0.0	33	56.9
Duplicate detection	0	0.0	14	24.1
Other	16	7.3	11	19.0
Total	218	100.0	58	100.0

The misses are not a resolution failure any more, they are an evidence failure. A target of fewer than ten pixels may retain only a few non-zero gradients after JPEG compression, and no architecture can recover what the sensor did not record. This is where a temporal filter, multiple frames or a better sensor would help more than another module. The false positives are a different problem: clutter contains small high-contrast structures that genuinely look like a distant UAV, and the attention branch reduces but does not remove them.

Two practical consequences follow. A deployed system should track across frames, because a target missed in one frame is often visible in the next and a track-level association would recover much of the 139. And a cheap second-stage classifier or a background-motion prior should cut most of the 33 clutter alarms, since those errors concentrate in static structures rather than being spread over the image.

7 Discussion

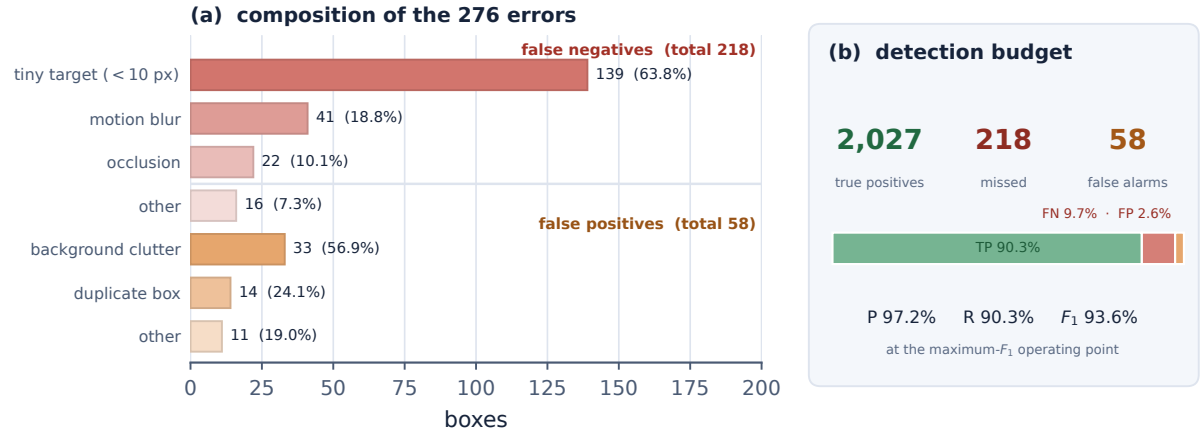


Figure 12. Error budget on the DUT Anti-UAV test set. (a) Composition of the 218 misses and 58 false alarms by cause. (b) Detection budget at the maximum-F_1 operating point: the three counts reproduce P, R, and F_1 exactly. Sub-10-pixel targets account for almost two thirds of the remaining misses.

7.1 Principal Findings

Four things come out of the experiments, and they hang together. Ultra-small UAV detection is a resolution problem more than a capacity problem: a 1.78M-parameter detector with a stride-4 path beats a 20M-parameter transformer and a 2.40M YOLO26n on the same benchmark, which is difficult to explain by capacity alone. The gains land where the design says they should, on the sub-20-pixel bins, while the large bins lose ground. In the cumulative sequence the largest single step on the strict metric is the head redesign, although that step bundles two edits. And the advantage survives domain shift, quantization and degraded imagery, which is evidence that it is not one lucky training run.

7.2 Why Resolution-Aware Co-Design Works

The argument is short enough to fit in a paragraph. A detector cannot localize what its finest feature map cannot represent. At stride 8 a ten-pixel UAV is 1.25 cells, and the regression branch is being asked to infer an extent from a single activation; at stride 4 it is 2.5 cells, which is a directional signal rather than a point. But the head alone would not do it. If the stages below have already aliased the high-frequency content away, a stride-4 map is just a smoother copy of a blurry image, which is why HWD matters: it keeps LH, HL and HH as separate channels and learns a 1×1 fusion, so the P2 features still carry edges and corners. Coordinate Attention then decides which rows and columns deserve weight, and BiFPN/CCFM move that detail upwards without letting the 256-channel deep features drown it. Four choices, four different jobs: the head supplies the sample, HWD supplies the signal, CA supplies the selectivity, the neck supplies the context. We did not remove each one in isolation, so this remains a reading of the design and of the cumulative sequence in Table 7 rather than a measured interaction.

The same argument accounts for the result we do not like. Without a P5 head there is no stride-32 prediction path, so anything wider than 32 pixels has to be regressed from P4 or below, and the deficit on mAP@0.5:0.95 grows with size: 6.5 points in the 32-64 pixel width bin, 11.0 in 64-96, 13.0 above 96. For long-range work that is acceptable, since the target is small by definition. For a system that also has to see a drone at fifty metres it is a real limitation, and the honest description is that we traded one end of the range for the other. Trading P4 for P5 costs 0.18M parameters and 0.2 points of aggregate mAP@0.5 (Table 8C), so a deployment that needs both ends could switch heads according to range; we did not train the P2+P3+P4+P5 variant, so we cannot say how much large-object accuracy such a switch would recover.

7.3 Accuracy-Efficiency Trade-off

At FP32 the model is about 20% slower than the baseline, and that is not an accident. The P2 feature map has four times the area of P3 and the head is memory-bound, so latency does not follow parameter count. Spending milliseconds at high resolution is the whole point of the design; the question is whether the budget still closes. On the RTX 3090 it does at FP16 (143 FPS) and at INT8 (221 FPS, 0.4 points of mAP@0.5 lost). On the Orin it closes at FP16 with 34 FPS, which is what R5 asked for, and INT8 leaves room for a second stream. Our recommendation for a field deployment is the FP16 engine: it keeps the accuracy, halves the memory and leaves headroom.

The comparison with Gao *et al.* [50] should be read with some care. Both methods use HWD, CCFM and a P2 head, and the 1.5-point advantage on the strict metric could come from the BiFPN, from the attention module, or from differences

in training that nobody documented. Without a reproduction under our protocol we cannot attribute it, so we treat it as supporting evidence and let the component-level checks in Table 8 carry the argument.

7.4 Limitations

One dataset, one class. Every controlled experiment runs on DUT Anti-UAV, which has a single UAV class in the visible spectrum. The cross-dataset results soften this but do not remove it, because those sets are used for transfer rather than for a full multi-class evaluation. A real C-UAV station has to separate UAVs from birds, light aircraft and plastic bags, and it has to cope with airframes it has never seen. Multi-class training and open-set recognition are the obvious next steps and we have not taken them.

No temporal information. The detector looks at one frame at a time. Since 63.8% of the remaining misses are sub-10-pixel targets and many of them are visible in the neighbouring frame, a tracking layer or a small recurrent aggregator would probably recover a good share of them. We kept the model frame-based on purpose, so that the architectural claim could be isolated, but that is a laboratory choice and not a deployment recommendation.

Visible spectrum. Night performance is only tested through synthetic darkening. Real thermal imagery has different statistics, and RGB-IR fusion needs a dual stream and a registration procedure. Work on reliability-guided fusion [20] and dual-stream UAV detection [52] is complementary to ours rather than competing with it.

Removing P5. As above: objects wider than 32 pixels pay for the P2 head. Applications that need a single model across the whole range would have to keep a stride-32 head; the closest configuration we trained, P2+P3+P5, trades P4 for P5 at 0.18M extra parameters, and we did not train P2+P3+P4+P5.

Regression head. The proposed model uses a 16-bin DFL head, whereas YOLO26 regresses the box directly. We did not run a DFL-versus-direct ablation, so we cannot separate the contribution of the resolution design from that of the regression parameterisation. The size-stratified gains are consistent with the resolution argument, but a reviewer should treat the head difference as a confound that the current experiments do not remove.

Label noise. The benchmark draws from many sources with different compression, and some boxes are tight while others include context. A three-pixel annotation error is nothing on a 300-pixel car and everything on a ten-pixel drone. We did not re-annotate, so the reported numbers include that noise; a cleaned test set would sharpen the estimate of the architectural gain, though we cannot say in which direction.

Hardware scope. Latency is measured on two devices. An FPGA or a mobile NPU will have different bottlenecks, and we would not be surprised if the wavelet transform and the 7×7 attention convolution behave quite differently there. Hardware-aware search over block widths is the natural remedy.

Width bins are not COCO categories. The six bins used throughout the paper are defined on the annotated box width, because width is what maps onto the sampling stride. They are deliberately not the COCO area categories: a box can be narrow and tall, and a width bin therefore does not partition the area ranges cleanly. The overall row of Table 9 is computed directly over the complete test split; it is not a weighted mean of the width bins, because AP is not linearly decomposable across subgroups. The COCO ranges are defined on box area instead, so a box that is narrow but tall can fall on the other side of an area boundary, and we do not report an area-based stratification.

The ablation is cumulative. Table 7 adds one component at a time, which answers the question the paper actually asks, namely what each step contributes once the previous ones are in place, but it does not separate a component's standalone effect from its interaction with the rest. Adding P2 to the bare baseline, or HWD to a model without Coordinate Attention, would give different numbers. A full factorial matrix over the six components, and the intermediate head configurations we did not train (P2+P3, P3+P4, P2+P3+P4+P5), is the experiment that would settle the question, and we have not run it.

One evaluation point. Precision, recall and F_1 are reported at the single operating point that maximises F_1 on the validation split. A different threshold would trade precision against recall, and a deployment with a fixed false-alarm budget would sit somewhere else on the same curve. The AP figures are threshold-free, which is why we lean on them for the comparative claims.

7.5 Threats to Validity

Internal validity. The obvious risk is hyperparameter bias, with the larger models under-tuned relative to ours. We searched the learning rate for every retrained detector, kept the epoch budget, augmentation and evaluation script

identical, and used the official YOLO26n implementation as the baseline. One asymmetry remains and we do not want to hide it: YOLO26 is designed for MuSGD with ProgLoss and STAL, whereas our common protocol trains every model with AdamW and a TAL assigner. The baseline is therefore not at its native best, and part of the measured gap could shrink under its own recipe. We chose a common optimizer because it is the only way to compare six families under one budget, and we report the constraint rather than presenting the baseline as a straw man. The one comparison we could not control is the quoted Gao *et al.* row, and we flag it wherever it appears.

External validity. The benchmark is mostly single-target frames, whereas a real scene may hold several UAVs and many distractors. Anti-UAV, Drone-vs-Bird and Det-Fly provide some evidence of transfer, but they do not cover every sensor, weather condition or airframe. Performance on a new site should be measured, not extrapolated.

Construct validity. mAP ranks detections; a C-UAV operator cares about detection range, false alarms per hour and time to first detection. A 4.5-point mAP gain does not translate linearly into any of those. The size-stratified table and the error decomposition are our attempt to bridge the gap, but a field trial with a tracker and a fixed false-alarm budget would say more.

Statistical validity. Ten seeds capture run-to-run variance, not variance from data sampling or hardware. The bootstrap intervals quantify the former. The effects here are large enough that more seeds are unlikely to reverse a conclusion, but the point estimates should still be read as approximate.

7.6 Broader Impact and Ethics

Detection of this kind has obvious defensive uses, in aviation safety, critical-infrastructure protection and privacy enforcement, and an equally obvious dual-use side: the same model can serve surveillance or targeting. We support responsible use and would insist on procedural safeguards, including human review of alerts and limits on how long video is retained. Two properties of the design are mildly reassuring from a privacy standpoint. It is small enough to run at the camera, so footage need not leave the site, and it detects rather than identifies, with no biometric component. The datasets used here are public and were collected for research; no new personal data were gathered.

7.7 Future Work

Four directions look worth pursuing. Temporal fusion is the most promising, since a light recurrent module or a track-level association would attack the 139 tiny-target misses directly. Multi-modal input, meaning RGB-IR with a registration-aware encoder, addresses the night case. Multi-class and open-set detection is the next discrimination frontier, and it is the one that decides whether a system can be deployed where birds are common. Hardware-aware architecture search is the least glamorous and possibly the most useful, because the wavelet transform and the P2 head will not cost the same on every accelerator. Distillation from a larger teacher and self-supervised pre-training on unlabelled aerial video are two further ways to improve the representation without paying for it at inference.

8 Conclusion

We started from a simple observation: at the stride used by the finest head of a standard YOLO detector, a long-range UAV is barely one feature cell wide, and no amount of capacity repairs a representation the target never occupied. BiFusion-YOLO is the response. A P2 head at stride 4 opens a high-resolution path, Haar wavelet downsampling keeps the high-frequency sub-bands alive along that path, Coordinate Attention recalibrates with row and column descriptors rather than a global average, and a weighted BiFPN with CCFM blocks carries the resulting detail across scales. Ghost convolution, re-parameterized RepVGG blocks and the deletion of the P5 head pay for the extra resolution in parameters, though not in FLOPs.

On DUT Anti-UAV the model reaches 97.2% precision, 90.3% recall, 95.6% mAP@0.5 and 66.4% mAP@0.5:0.95 with 1.78M parameters, which is 4.3, 5.0, 4.5 and 5.6 points above YOLO26n, with 25.7% fewer parameters than the 2.40M baseline and 34.5% more FLOPs. For targets narrower than ten pixels, mAP@0.5:0.95 rises from 38.5% to 54.1%, and the 10–20 and 20–32 pixel bins gain 10.3 and 4.7 points. The detector runs at 89 FPS in FP32 on an RTX 3090 and 34 FPS in FP16 on a Jetson Orin, and it keeps its advantage under INT8 quantization, on three other datasets and under the usual image degradations.

The limits are equally clear. Objects wider than 32 pixels lose between 6.5 and 13.0 points of mAP@0.5:0.95 once the stride-32 head is gone. The evaluation is frame-based and covers a single visible-spectrum dataset, with no thermal imagery, and the ablation is cumulative rather than factorial. Those facts define the agenda we would follow next: aggregate evidence over time, add a second sensing modality for the night, learn to tell a drone from a bird, and search the architecture against the accelerator that will actually run it. The broader lesson we would like to leave is not about

any single module. It is that the resolution budget of a detector should be spent where the targets are, and that this decision is worth making before choosing which blocks to stack.

Reproducibility Statement

The release that accompanies the published version of this paper is designed to regenerate everything reported here: the training and evaluation code, the exact train/validation/test image lists, the model configuration, the augmentation and loss settings, the ten seeds used for the statistics, the evaluation script with its IoU and confidence settings, the scripts for the significance tests, and the Python scripts that redraw every figure from the numbers in the tables. A dataset-audit script recomputes the counts of Table 4 from the label files, including positive and negative images, malformed annotations and duplicate detection, so the split statistics can be checked rather than trusted. The datasets are public (Section 5). The code and configuration files are publicly available at `https://github.com/mohamedalieladl/BIFUSION-YOLO`; trained weights and TensorRT engines are available from the authors on reasonable request. The full training recipe is in Table 5, and the latency-measurement protocol is in Section 5.2.

Data and Code Availability

The DUT Anti-UAV dataset is publicly available from its original authors [12]. The Anti-UAV benchmark [11], Drone-vs-Bird [13] and Det-Fly [63] are also public under their respective licenses. The source code, configuration files, evaluation scripts and figure-generation code for BiFusion-YOLO are publicly available at `https://github.com/mohamedalieladl/BIFUSION-YOLO`; trained weights and TensorRT engines are available from the authors on reasonable request.

Author Contributions

Mohamed Ali: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing (original draft), writing (review and editing), visualization. **Mahmoud M. Safan:** Conceptualization, methodology, supervision, writing (review and editing), project administration. **Sarah M. Ayyad:** Methodology, supervision, writing (review and editing), resources. All authors have read and agreed to the published version of the manuscript.

Funding

This research received no external funding.

Competing Interests

The authors declare no competing interests. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Ethics Statement

This study used publicly available datasets collected for research purposes. No new data were collected from human participants or animals. The authors support the responsible use of counter-UAV technology and recommend human review of automated alerts in operational deployments.

Acknowledgments

The authors thank the maintainers of the DUT Anti-UAV, Anti-UAV, Drone-vs-Bird and Det-Fly datasets for making their data publicly available, and the Ultralytics team for the open-source implementation used as the baseline.

A Notation

Table 14 lists the symbols used throughout the paper.

Table 14. Notation.

Symbol	Meaning
$I \in \mathbb{R}^{H\times W\times 3}$	Input RGB image
$\mathcal{B} = \{(b_i, c_i)\}_{i=1}^{M}$	Ground-truth boxes and class labels
$b_i = (x_i, y_i, w_i, h_i)$	Box centre, width, and height
$\hat{\mathcal{B}} = \{(\hat{b}_j, \hat{s}_j)\}_{j=1}^{N}$	Predicted boxes and confidence scores
F_ℓ	Backbone feature map at level ℓ
$\tilde{F}_\ell$	Fused feature map at level ℓ
s_ℓ	Detection stride at level ℓ
$n_\ell(w)$	Number of feature cells spanned by width w at level ℓ
ρ_i	Object-to-image area ratio
a_i	Object area $w_i h_i$
P, R, F_1	Precision, recall, and their harmonic mean
AP, mAP	Average precision and mean average precision
N_{TP}, N_{FP}, N_{FN}	True positives, false positives, false negatives
$\mathcal{L}_{CIoU}, \mathcal{L}_{BCE}, \mathcal{L}_{DFL}$	Box, classification, and distribution-focal losses
$\lambda_{box}, \lambda_{cls}, \lambda_{dfl}$	Loss weights
$w_{i,j}$	Learnable BiFPN fusion weight
M_s	Spatial-attention map
α, β	Task-aligned assigner exponents
d_z	Cohen's standardized effect size for paired data

References

[1] Hazim Shakhatreh, Ahmad H. Sawalmeh, Ala Al-Fuqaha, Zuochao Dou, Eyad Almaita, Issa Khalil, Nasser S. Othman, Abdallah Khreishah, and Mohsen Guizani. Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges. *IEEE Access*, 7:48572-48634, 2019.

[2] Yassine Mekdad, Ahcène Aris, Leonardo Babun, Abdeslam El Fergougui, Mauro Conti, Riccardo Lazzeretti, and A. Selcuk Uluagac. A survey on security and privacy issues of UAVs. *Computer Networks*, 224:109626, 2023.

[3] Xiufang Shi, Chaoqun Yang, Weige Xie, Chao Liang, Zhaoyang Shi, and Jiming Chen. Anti-drone system with multiple surveillance technologies: Architecture, implementation, and challenges. *IEEE Communications Magazine*, 56(4):68-74, 2018.

[4] BBC News. Gatwick drone attack: 50 officers still deployed, 2018. [Online]. Available: https://www.bbc.com/news/uk-england-sussex-46623754.

[5] Mohammad Jahangir and Christopher J. Baker. Extended target recognition using micro-Doppler signals. *IET Radar, Sonar & Navigation*, 6(2):36-45, 2012.

[6] Kyung N. Kim and Chang W. Lim. A practical limit of detection for low RCS targets with a commercial S-band radar. *IEEE Transactions on Aerospace and Electronic Systems*, 55(4):2026-2034, 2019.

[7] J. J. M. de Wit et al. RF-based passive bistatic radar for short range mini-UAV detection. In *IEEE Radar Conference*, pages 1-6, 2020.

[8] Arjun Sathyan et al. An acoustic system for small UAV detection. In *IEEE International Conference on Advanced Video and Signal-Based Surveillance (AVSS)*, pages 1-6, 2015.

[9] Muhammad Saqib et al. A study on detecting drones using deep convolutional neural networks. In *IEEE International Conference on Advanced Video and Signal-Based Surveillance (AVSS)*, pages 1-5, 2017.

[10] Hui Liu et al. Detection and tracking of infrared small target with complex background based on deep learning. *Applied Sciences*, 9(17):3510, 2019.

[11] Nan Jiang, Kuiran Wang, Xiaoke Peng, Xuehui Yu, Qiang Wang, Junliang Xing, Guorong Li, Jianan Ye, Zonghao Li, Xiaobin Xu, et al. Anti-UAV: A large multi-modal benchmark for UAV tracking. *arXiv preprint arXiv:2101.08466*, 2021.

[12] Jie Zhao, Jingshu Zhang, Defang Li, and Dawei Wang. Vision-based anti-UAV detection and tracking. *IEEE Transactions on Intelligent Transportation Systems*, 23(12):25323-25334, 2022.

[13] Angelo Coluccia, Alessio Fascista, Arne Schumann, Lars Sommer, Marian Ghenescu, Tomas Piatrik, Geert De Cubber, et al. Drone-vs-bird detection challenge at IEEE AVSS2021. In *IEEE International Conference on Advanced Video and Signal-Based Surveillance (AVSS)*, pages 1-7, 2021.

[14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779-788, 2016.

[15] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[16] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[17] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7464-7475, 2023.

[18] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. YOLOv10: Real-time end-to-end object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[19] Ultralytics. YOLO26: Real-time object detection. https://docs.ultralytics.com/models/yolo26, 2026. Model documentation, accessed November 2026.

[20] Liming Hou, Yueping Peng, Hexiang Hao, Ji Wang, Xuekai Zhang, Wei Tang, Zecong Ye, Xin Ying, and Yubo He. LER-YOLO: Reliability-aware expert routing for misaligned RGB-infrared UAV detection. *arXiv preprint arXiv:2605.20667*, 2026.

[21] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 91-99, 2015.

[22] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21-37, 2016.

[23] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2980-2988, 2017.

[24] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. FCOS: A simple and strong anchor-free object detector. In *IEEE International Conference on Computer Vision (ICCV)*, pages 9627-9636, 2019.

[25] Glenn Jocher. YOLOv5, 2020. Ultralytics GitHub repository.

[26] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. YOLOv6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.

[27] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. YOLOv8, 2023. Ultralytics.

[28] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. YOLOv9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*, 2024.

[29] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, pages 213-229, 2020.

[30] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2021.

[31] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. DETRs beat YOLOs on real-time object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965-16974, 2024.

[32] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. Ultralytics YOLO26: Unified real-time end-to-end vision models. *arXiv preprint arXiv:2606.03748*, 2026.

[33] Ranjan Sapkota and Manoj Karkee. Ultralytics YOLO evolution: An overview of YOLO27, YOLO26, YOLO11, YOLOv8, and YOLOv5 object detectors for computer vision and pattern recognition. *arXiv preprint arXiv:2510.09653*, 2025.

[34] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117-2125, 2017.

[35] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. Path aggregation network for instance segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8759-8768, 2018.

[36] Golnaz Ghiasi, Tsung-Yi Lin, and Quoc V. Le. NAS-FPN: Learning scalable feature pyramid architecture for object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7036-7045, 2019.

[37] Mingxing Tan, Ruoming Pang, and Quoc V. Le. EfficientDet: Scalable and efficient object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10781-10790, 2020.

[38] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132-7141, 2018.

[39] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. CBAM: Convolutional block attention module. In *European Conference on Computer Vision (ECCV)*, pages 3-19, 2018.

[40] Qilong Wang, Banggu Wu, Pengfei Zhu, Peihua Li, Wangmeng Zuo, and Qinghua Hu. ECA-Net: Efficient channel attention for deep convolutional neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11534-11542, 2020.

[41] Lingxiao Yang, Ru-Yuan Zhang, Lishan Li, and Xiaohua Xie. SimAM: A simple, parameter-free attention module for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, pages 11863-11874, 2021.

[42] Qibin Hou, Daquan Zhou, and Jiashi Feng. Coordinate attention for efficient mobile network design. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13713-13722, 2021.

[43] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510-4520, 2018.

[44] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. ShuffleNet V2: Practical guidelines for efficient CNN architecture design. In *European Conference on Computer Vision (ECCV)*, pages 116-131, 2018.

[45] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. GhostNet: More features from cheap operations. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1580-1589, 2020.

[46] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. RepVGG: Making VGG-style ConvNets great again. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13733-13742, 2021.

[47] Mate Kisantal, Zbigniew Wojna, Jakub Murawski, Jacek Naruniec, and Kyunghyun Cho. Augmentation for small object detection. In *International Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2019.

[48] Amir Zamani and Zeinab Abedini. Optimizing data augmentation for real-time small UAV detection: A lightweight context-aware approach. *arXiv preprint arXiv:2604.19999*, 2026.

[49] Guoping Xu, Wenti Liao, Xin Zhang, Cheng Li, Xuedan He, and Xiaolin Wu. Haar wavelet downsampling: A simple but effective downsampling module for semantic segmentation. *Pattern Recognition*, 143:109819, 2023.

[50] Yunlong Gao, Yibing Xin, Huan Yang, and Yongjuan Wang. A lightweight anti-unmanned aerial vehicle detection method based on improved YOLOv11. *Drones*, 9(1):11, 2025.

[51] Ami Pandat, Punna Rajasekhar, Gopika Vinod, and Rohit Shukla. YolovN-CBi: A lightweight and efficient architecture for real-time detection of small UAVs. *arXiv preprint arXiv:2512.18046*, 2026.

[52] Shahriar Kabir, Istiak Ahmmed Rifti, H. M. Shadman Tabib, Mushfiqur Rahman, Sadatul Islam Sadi, Hasnaen Adil, Ahmed Mahir Sultan Rumi, and Ch. Md. Rakin Haider. SpectraSentinel: Lightweight dual-stream real-time drone detection, tracking and payload identification. *arXiv preprint arXiv:2507.22650*, 2025.

[53] Amir Zamani and Zeinab Ghasemi-Naraghi. CSCWD: Cross-scale channel-wise knowledge distillation for lightweight tiny object detection on edge devices. *arXiv preprint arXiv:2609.30395*, 2026.

[54] Pengyu Chen, Haotian Sa, Yiwei Hu, Yuhan Cheng, and Junbo Wang. SDD-YOLO: A small-target detection framework for ground-to-air anti-UAV surveillance with edge-efficient deployment. *arXiv preprint arXiv:2603.25218*, 2026.

[55] Juanqin Liu, Leonardo Plotegher, Eloy Roura, Cristino de Souza Junior, and Shaoming He. Real-time detection for small UAVs: Combining YOLO and multi-frame motion analysis. *arXiv preprint arXiv:2411.02582*, 2024.

[56] Rayson Laroca, Marcelo dos Santos, and David Menotti. Improving small drone detection through multi-scale processing and data augmentation. *arXiv preprint arXiv:2504.19347*, 2025.

[57] Tamara R. Lenhard, Andreas Weinmann, Stefan Jäger, and Tobias Koch. YOLO-FEDER FusionNet: A novel deep learning architecture for drone detection. In *IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, pages 1-5, 2024.

[58] Tamara R. Lenhard, Andreas Weinmann, and Tobias Koch. Performance optimization of YOLO-FEDER FusionNet for robust drone detection in visually complex environments. *arXiv preprint arXiv:2509.14012*, 2025.

[59] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740-755, 2014.

[60] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 21002-21012, 2020.

[61] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R. Scott, and Weilin Huang. TOOD: Task-aligned one-stage object detection. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3490-3499, 2021.

[62] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-IoU loss: Faster and better learning for bounding box regression. In *AAAI Conference on Artificial Intelligence*, volume 34, pages 12993-13000, 2020.

[63] Yuxin Zheng et al. Det-Fly: Detecting flying objects in aerial images. *IEEE Transactions on Intelligent Transportation Systems*, 2021.

[64] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[65] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1-30, 2006.

[66] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895-1923, 1998.

[67] Yunjie Tian, Qixiang Ye, and David Doermann. YOLOv12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524*, 2025.

[68] Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 658-666, 2019.

[69] Yi-Fan Zhang, Weiqiang Ren, Zhang Zhang, Zhen Jia, Liang Wang, and Tieniu Tan. Focal and efficient IoU loss for accurate bounding box regression. *Neurocomputing*, 506:146-157, 2022.